OpenClaw

AppClaw — Hermes Agent + Personalization Layer

Complete Swift Source Code

21 source files

File	Description
AppClawApp.swift	@main entry point, BGTask, onboarding gate
BearLogoView.swift	OpenClaw bear head logo (SwiftUI Path)
ChatView.swift	Streaming chat UI (OpenClaw theme)
HermesAgentHarness.swift	Agent roles, harness, typed event stream
HermesAutomation.swift	Spending tracker + app URL automation
HermesContextTracker.swift	Sliding-window topic & intent detector
HermesCronScheduler.swift	Cron jobs + NL schedule parser
HermesDreamEngine.swift	Autodream consolidation (5 phases)
HermesIntegration.swift	High-level logging API
HermesInterestEngine.swift	Interest-based notification scheduling
HermesKairos.swift	15s budget proactive observer
HermesLLMClient.swift	Foundation Models + Claude API routing
HermesMemory.swift	On-device memory store + MemoryIndex
HermesPersonality.swift	Persona prompt builder + affirmations
HermesPrivacyGate.swift	Guideline 5.1.1 consent gate
HermesProactiveEngine.swift	Always-on suggestion engine
HermesSessionState.swift	Crash-safe state, workflow, token budget
HermesTheme.swift	OpenClaw colour palette & typography
HermesToolRegistry.swift	Metadata-first registry + 18-module security
OnboardingView.swift	6-step animated onboarding flow
UserPersona.swift	User profile, interests, companion mode

◆ AppClawApp.swift (76 lines)

```swift
import SwiftUI
import BackgroundTasks

@main
struct AppClawApp: App {

@StateObject private var appState = AppState()

init() {
// Register BGProcessingTask as early as possible (before first scene)
HermesDreamEngine.shared.registerBackgroundTask()
}

var body: some Scene {
WindowGroup {
RootView()
.environmentObject(appState)
.preferredColorScheme(.dark) // always dark — OpenClaw is a dark-first app
}
}
}

// MARK: - AppState

/// Single source of truth for app-level state shared across views.
@MainActor
final class AppState: ObservableObject {
@Published var onboardingComplete: Bool
@Published var privacyState: PrivacyConsentState = .notAsked

init() {
onboardingComplete = UserDefaults.standard.bool(forKey: "onboardingComplete")
}

func completeOnboarding() {
onboardingComplete = true
UserDefaults.standard.set(true, forKey: "onboardingComplete")
}
}

// MARK: - RootView

/// Gates between onboarding and the main chat interface.
struct RootView: View {
@EnvironmentObject private var appState: AppState
@State private var hermesReady = false

var body: some View {
Group {
if !appState.onboardingComplete {
OnboardingView()
} else {
ChatView()
}
}
.task {
// Load session state and start Hermes subsystems
await HermesSessionState.shared.loadFromDisk()
await HermesPrivacyGate.shared.configureHermesIfReady()
hermesReady = true
}
.onReceive(NotificationCenter.default.publisher(for: UIApplication.didEnterBackgroundNotification)) { _ in
Task {
await HermesIntegration.shared.logSessionEnd()
HermesDreamEngine.shared.scheduleNextDream()
await HermesKairos.shared.pause()
}
}
.onReceive(NotificationCenter.default.publisher(for: UIApplication.willEnterForegroundNotification)) { _ in
Task {
await HermesPrivacyGate.shared.configureHermesIfReady()
await HermesProactiveEngine.shared.refresh()
}
}
}
```

```
}
```

◆ BearLogoView.swift (253 lines)

```swift
import SwiftUI

// MARK: - OpenClaw Bear Head Logo
//
// A cute geometric bear head built entirely from SwiftUI Shapes —
// no external assets needed. Scales cleanly from 24 pt (nav bar icon)
// up to 512 pt (App Store icon).
//
// Colour scheme follows the OpenClaw theme: midnight navy background,
// amber-claw gradient, electric-blue accent eyes.

// MARK: - Bear shape primitives

/// Full bear head silhouette (face + round ears).
struct BearHeadShape: Shape {
func path(in rect: CGRect) -> Path {
let w = rect.width, h = rect.height
var p = Path()

// Main circular face
let faceR = w * 0.42
let faceC = CGPoint(x: w * 0.50, y: h * 0.54)
p.addEllipse(in: CGRect(
x: faceC.x - faceR, y: faceC.y - faceR,
width: faceR * 2, height: faceR * 2
))

// Left ear
let earR = w * 0.18
p.addEllipse(in: CGRect(
x: w * 0.12, y: h * 0.06,
width: earR * 2, height: earR * 2
))

// Right ear
p.addEllipse(in: CGRect(
x: w * 0.70, y: h * 0.06,
width: earR * 2, height: earR * 2
))

return p
}
}

/// Inner ear circles.
struct BearInnerEarShape: Shape {
func path(in rect: CGRect) -> Path {
let w = rect.width, h = rect.height
var p = Path()
let r = w * 0.10

// Left inner ear
p.addEllipse(in: CGRect(x: w * 0.18, y: h * 0.12, width: r * 2, height: r * 2))
// Right inner ear
p.addEllipse(in: CGRect(x: w * 0.72, y: h * 0.12, width: r * 2, height: r * 2))
return p
}
}

/// Snout area — slightly lighter oval below the eyes.
struct BearSnoutShape: Shape {
func path(in rect: CGRect) -> Path {
let w = rect.width, h = rect.height
var p = Path()
let sw = w * 0.32, sh = h * 0.20
p.addEllipse(in: CGRect(
x: (w - sw) / 2, y: h * 0.60,
width: sw, height: sh
))
return p
}
}

// MARK: - BearLogoView

```

```
/// Composited bear head with OpenClaw theming.
/// - Parameters:
/// - size: Bounding square in points (default 64).
/// - showBackground: If true, draws a rounded-rect card background.
struct BearLogoView: View {
var size: CGFloat = 64
var showBackground: Bool = true

var body: some View {
ZStack {
// Optional card background
if showBackground {
RoundedRectangle(cornerRadius: size * 0.22)
.fill(
LinearGradient(
colors: [Color(hex: "#1A2233"), Color.OC.background],
startPoint: .topLeading,
endPoint: .bottomTrailing
)
)
.frame(width: size, height: size)
.shadow(color: Color.OC.accent.opacity(0.3), radius: size * 0.12, y: size * 0.04)
}

// Bear head silhouette — amber-claw gradient
BearHeadShape()
.fill(
LinearGradient(
colors: [Color(hex: "#C8862A"), Color(hex: "#FF9F0A"), Color(hex: "#E8A030")],
startPoint: .topLeading,
endPoint: .bottomTrailing
)
)
.frame(width: size * 0.82, height: size * 0.82)

// Inner ears — darker amber
BearInnerEarShape()
.fill(Color(hex: "#A06820"))
.frame(width: size * 0.82, height: size * 0.82)

// Snout — cream
BearSnoutShape()
.fill(Color(hex: "#F0D090"))
.frame(width: size * 0.82, height: size * 0.82)

// Eyes — electric blue with specular dot
bearEyes

// Nose — small dark oval on snout
Ellipse()
.fill(Color(hex: "#3A2010"))
.frame(width: size * 0.10, height: size * 0.065)
.offset(y: size * 0.185)

// Subtle "claw mark" overlay — three thin diagonal lines at bottom-right
if size >= 40 {
clawMarkOverlay
}
}
.frame(width: size, height: size)
}

// MARK: - Eyes

@ViewBuilder
private var bearEyes: some View {
let eyeR = size * 0.068
let eyeOffX = size * 0.135
let eyeOffY = size * 0.04

ZStack {
// Left eye white
Circle().fill(Color.white)
.frame(width: eyeR * 2.2, height: eyeR * 2.2)
.offset(x: -eyeOffX, y: -eyeOffY)
// Left iris — electric blue
Circle()
```

```
.fill(
RadialGradient(
colors: [Color.OC.primary, Color(hex: "#0055CC")],
center: .center, startRadius: 0, endRadius: eyeR
)
)
.frame(width: eyeR * 2, height: eyeR * 2)
.offset(x: -eyeOffX, y: -eyeOffY)
// Left specular
Circle().fill(Color.white.opacity(0.7))
.frame(width: eyeR * 0.55, height: eyeR * 0.55)
.offset(x: -eyeOffX + eyeR * 0.35, y: -eyeOffY - eyeR * 0.35)

// Right eye white
Circle().fill(Color.white)
.frame(width: eyeR * 2.2, height: eyeR * 2.2)
.offset(x: eyeOffX, y: -eyeOffY)
// Right iris
Circle()
.fill(
RadialGradient(
colors: [Color.OC.primary, Color(hex: "#0055CC")],
center: .center, startRadius: 0, endRadius: eyeR
)
)
.frame(width: eyeR * 2, height: eyeR * 2)
.offset(x: eyeOffX, y: -eyeOffY)
// Right specular
Circle().fill(Color.white.opacity(0.7))
.frame(width: eyeR * 0.55, height: eyeR * 0.55)
.offset(x: eyeOffX + eyeR * 0.35, y: -eyeOffY - eyeR * 0.35)
}
}

// MARK: - Claw mark

@ViewBuilder
private var clawMarkOverlay: some View {
let s = size
// Bug fix #1: use the Canvas closure's own `canvasSize` instead of the
// captured `s` — avoids misalignment when layout resolves a different size.
Canvas { ctx, canvasSize in
let w = canvasSize.width, h = canvasSize.height
let offX = w * 0.26
let offY = h * 0.22
let len = h * 0.18
let gap = w * 0.055
for i in 0..<3 {
let dx = CGFloat(i) * gap
var line = Path()
line.move(to: CGPoint(x: w / 2 + offX + dx, y: h / 2 + offY))
line.addLine(to: CGPoint(x: w / 2 + offX + dx + len * 0.5, y: h / 2 + offY + len))
ctx.stroke(line,
with: .color(Color.OC.accent.opacity(0.45)),
style: StrokeStyle(lineWidth: w * 0.022, lineCap: .round))
}
}
.frame(width: size, height: size)
.blendMode(.overlay)
}
}

// MARK: - App icon variant (no background card, for Asset Catalog)

struct BearIconView: View {
var size: CGFloat = 1024
var body: some View {
ZStack {
// Solid background for App Store icon
RoundedRectangle(cornerRadius: size * 0.2237) // Apple icon corner formula
.fill(
LinearGradient(
colors: [Color(hex: "#131A25"), Color(hex: "#0D1117")],
startPoint: .top, endPoint: .bottom
)
)
BearLogoView(size: size * 0.75, showBackground: false)
```

```
}
.frame(width: size, height: size)
}
}

// MARK: - Preview

#if DEBUG
#Preview("Logo sizes") {
HStack(spacing: 20) {
ForEach([24, 44, 64, 96], id: \.self) { sz in
BearLogoView(size: CGFloat(sz))
}
}
.padding(24)
.background(Color.OC.background)
}

#Preview("App icon") {
BearIconView(size: 256)
.padding(20)
.background(Color.OC.background)
}
#endif
```

◆ ChatView.swift (809 lines)

```swift
import SwiftUI
import Combine

// MARK: - ChatMessage

struct ChatMessage: Identifiable, Equatable {
let id: UUID
let role: MessageRole
var text: String
let timestamp: Date
var isStreaming: Bool

enum MessageRole { case user, assistant, system }

init(id: UUID = UUID(), role: MessageRole, text: String,
timestamp: Date = Date(), isStreaming: Bool = false) {
self.id = id; self.role = role; self.text = text
self.timestamp = timestamp; self.isStreaming = isStreaming
}
}

// MARK: - ChatViewModel

@MainActor
final class ChatViewModel: ObservableObject {
@Published var messages: [ChatMessage] = []
@Published var inputText: String = ""
@Published var isTyping: Bool = false
@Published var suggestions: [String] = []
@Published var affirmation: String? = nil
@Published var showAffirmation: Bool = false
@Published var quickActions: [(title: String, icon: String, action: () -> Void)] = []

private var streamingID: UUID?
private var suggestionTask: Task<Void, Never>?
private let persona: UserPersona
private let sessionId: String = UUID().uuidString

init(persona: UserPersona) {
self.persona = persona
Task { await setup() }
}

// MARK: - Setup

private func setup() async {
// Daily affirmation
let aff = await HermesPersonality.shared.todaysAffirmation(for: persona)
let lastShown = UserDefaults.standard.object(forKey: "lastAffirmationDate") as? Date
let today = Calendar.current.startOfDay(for: Date())
if lastShown == nil || Calendar.current.startOfDay(for: lastShown!) < today {
affirmation = aff
showAffirmation = true
}

// Greeting if first launch today
if messages.isEmpty {
let name = persona.userName.isEmpty ? "" : " \(persona.userName)"
let hour = Calendar.current.component(.hour, from: Date())
let greeting: String
switch hour {
case 5..<12: greeting = "Good morning\(name)! ■■ What's on your mind?"
case 12..<17: greeting = "Hey\(name)! ■ How's your afternoon going?"
case 17..<21: greeting = "Good evening\(name)! ■ How was your day?"
default: greeting = "Hey\(name) ■ Up late? What's going on?"
}
messages.append(ChatMessage(role: .assistant, text: greeting))
}

// Load suggestions
await refreshSuggestions()

// Build quick actions
buildQuickActions()
}
```

```swift

// MARK: - Send message

func send() async {
let text = inputText.trimmingCharacters(in: .whitespacesAndNewlines)
guard !text.isEmpty else { return }
inputText = ""

// Append user message
messages.append(ChatMessage(role: .user, text: text))

// Snapshot history now (before assistant placeholder is added)
let history = buildHistory()

// Log to memory
await HermesIntegration.shared.logUserMessage(text, in: sessionId)
Kairos.shared.userDidAct()

// Learn facts and interests from this message
learnFromMessage(text)

// Check for automation intent
if let task = await HermesAutomation.shared.detectTask(from: text) {
await HermesAutomation.shared.saveTask(task)
}

// Detect cron schedule intent
let lower = text.lowercased()
if lower.contains("remind") || lower.contains("every day") ||
lower.contains("schedule") || lower.contains("every week") {
if let schedule = HermesCronScheduler.parseSchedule(from: text) {
let job = CronJob(title: String(text.prefix(50)), body: text, schedule: schedule)
await HermesCronScheduler.shared.add(job)
}
}

// Stream assistant response
await streamResponse(history: history)

// Refresh suggestions in background
suggestionTask?.cancel()
suggestionTask = Task {
try? await Task.sleep(nanoseconds: 1_000_000_000)
await refreshSuggestions()
}
}

private func streamResponse(history: [(role: String, content: String)]) async {
isTyping = true
let msgID = UUID()
streamingID = msgID
messages.append(ChatMessage(id: msgID, role: .assistant, text: "", isStreaming: true))

// Build LLM request from pre-captured history
let request = LLMRequest(
systemPrompt: await buildPersonaSystemPrompt(),
messages: history.map { LLMMessage(role: $0.role == "user" ? .user : .assistant, content: $0.content) },
tools: [],
maxTokens: 1024,
role: .execute
)

// Stream tokens — look up message by UUID each time to avoid stale index
let capturedID = msgID
do {
let response = try await HermesLLMClient.shared.complete(
request: request,
stream: { [weak self] token in
Task { @MainActor [weak self] in
guard let self, self.streamingID == capturedID,
let i = self.messages.firstIndex(where: { $0.id == capturedID })
else { return }
self.messages[i].text += token
}
}
)
// Non-streaming providers return full text in response.content
```

```
if let i = messages.firstIndex(where: { $0.id == capturedID }) {
if messages[i].text.isEmpty { messages[i].text = response.content }
messages[i].isStreaming = false
}
} catch {
if let i = messages.firstIndex(where: { $0.id == capturedID }) {
messages[i].text = "Sorry, I had trouble responding. Try again?"
messages[i].isStreaming = false
}
}

streamingID = nil
isTyping = false

let finalText = messages.first(where: { $0.id == capturedID })?.text ?? ""
await HermesIntegration.shared.logAssistantResponse(finalText)
learnFromAssistantMessage(finalText)
}

private func buildPersonaSystemPrompt() async -> String {
await HermesPersonality.shared.buildPersonaPrompt(for: persona)
}

private func buildHistory() -> [(role: String, content: String)] {
messages.suffix(20).compactMap { msg -> (role: String, content: String)? in
switch msg.role {
case .user: return (role: "user", content: msg.text)
case .assistant: return (role: "assistant", content: msg.text)
case .system: return nil
}
}
}

// MARK: - Learning

private func learnFromMessage(_ text: String) {
let facts = HermesPersonality.shared.extractFactsSync(from: text, persona: persona)
for (key, value) in facts {
persona.learn(key: key, value: value)
}
Task { @MainActor in
let interests = await HermesInterestEngine.shared.detectInterests(in: text)
for interest in interests {
if !self.persona.interests.contains(where: { $0.id == interest.id }) {
self.persona.interests.append(interest)
}
}
self.persona.save()
}
}

private func learnFromAssistantMessage(_ text: String) {
// Extract facts the assistant may have stated about the user
let facts = HermesPersonality.shared.extractFactsSync(from: text, persona: persona)
for (key, value) in facts { persona.learn(key: key, value: value) }
}

// MARK: - Suggestions

private func refreshSuggestions() async {
let raw = await HermesIntegration.shared.pollSuggestions()
suggestions = raw.prefix(4).map { $0.title }
}

// MARK: - Quick actions

private func buildQuickActions() {
quickActions = [
(title: "Log Spending", icon: "dollarsign.circle.fill", action: {
Task { @MainActor in
self.inputText = "I want to log some spending"
}
}),
(title: "My Schedule", icon: "calendar.badge.clock", action: {
Task { @MainActor in
self.inputText = "What do I have coming up?"
}
```

```
}),
(title: "Open App", icon: "apps.iphone", action: {
Task { @MainActor in
self.inputText = "Open Starbucks for me"
}
}),
(title: "Remind Me", icon: "bell.fill", action: {
Task { @MainActor in
self.inputText = "Remind me to "
}
}),
]
}

// MARK: - Dismiss affirmation

func dismissAffirmation() {
withAnimation { showAffirmation = false }
UserDefaults.standard.set(Date(), forKey: "lastAffirmationDate")
}
}

// MARK: - HermesPersonality sync helper (for use on MainActor)

extension HermesPersonality {
/// Synchronous wrapper — safe to call from non-async context on MainActor.
nonisolated func extractFactsSync(from text: String, persona: UserPersona) -> [String: String] {
// We can't call actor methods synchronously, so replicate the logic here
var facts: [String: String] = [:]
let lower = text.lowercased()
let nbaTeams = ["lakers","celtics","warriors","bulls","nets","knicks","heat","spurs","bucks"]
let nflTeams = ["chiefs","patriots","cowboys","packers","eagles","49ers","ravens","broncos"]
for team in nbaTeams where lower.contains(team) { facts["favorite_nba_team"] = team.capitalized }
for team in nflTeams where lower.contains(team) { facts["favorite_nfl_team"] = team.capitalized }
if lower.contains("starbucks") { facts["likes_starbucks"] = "true" }
if lower.contains("pizza") { facts["likes_pizza"] = "true" }
if lower.contains("gym") || lower.contains("workout") { facts["is_active"] = "true" }
if lower.contains("work from home") || lower.contains("wfh") { facts["works_from_home"] = "true" }
return facts
}
}

// MARK: - ChatView

struct ChatView: View {
@ObservedObject var persona: UserPersona
@StateObject private var vm: ChatViewModel
@Namespace private var bottomID
@State private var showSettings = false
@State private var showAutomation = false

init(persona: UserPersona) {
self.persona = persona
_vm = StateObject(wrappedValue: ChatViewModel(persona: persona))
}

var body: some View {
NavigationStack {
ZStack {
Color.OC.background.ignoresSafeArea()

VStack(spacing: 0) {
// Affirmation banner
if vm.showAffirmation, let aff = vm.affirmation {
AffirmationBanner(text: aff, onDismiss: vm.dismissAffirmation)
.transition(.move(edge: .top).combined(with: .opacity))
}

// Suggestion chips
if !vm.suggestions.isEmpty {
SuggestionChipsView(suggestions: vm.suggestions) { chip in
vm.inputText = chip
Task { await vm.send() }
}
}

// Message list
```

```
ScrollViewReader { proxy in
ScrollView {
LazyVStack(spacing: 12) {
ForEach(vm.messages) { msg in
MessageBubble(message: msg, persona: persona)
.id(msg.id)
}
if vm.isTyping {
TypingIndicator(name: persona.assistantName)
}
Color.clear.frame(height: 1).id("bottom")
}
.padding(.horizontal, 16)
.padding(.vertical, 12)
}
.onChange(of: vm.messages.count) { _ in
withAnimation(.easeOut(duration: 0.25)) {
proxy.scrollTo("bottom", anchor: .bottom)
}
}
.onChange(of: vm.isTyping) { _ in
withAnimation(.easeOut(duration: 0.25)) {
proxy.scrollTo("bottom", anchor: .bottom)
}
}
}

// Quick actions row
QuickActionsBar(actions: vm.quickActions)

// Input bar
InputBar(text: $vm.inputText) {
Task { await vm.send() }
}
}
}
.navigationBarTitleDisplayMode(.inline)
.toolbar {
ToolbarItem(placement: .navigationBarLeading) {
HStack(spacing: 8) {
BearLogoView(size: 32)
VStack(alignment: .leading, spacing: 0) {
Text(persona.assistantName.isEmpty ? "Claw" : persona.assistantName)
.font(OCFont.headline)
.foregroundColor(Color.OC.primaryText)
Text("Always here for you")
.font(OCFont.caption)
.foregroundColor(Color.OC.secondaryText)
}
}
}
ToolbarItem(placement: .navigationBarTrailing) {
Button {
showSettings = true
} label: {
Image(systemName: "gearshape.fill")
.foregroundColor(Color.OC.secondaryText)
.font(.system(size: 18))
}
}
}
.sheet(isPresented: $showSettings) {
SettingsView(persona: persona)
}
}
.preferredColorScheme(.dark)
}
}

// MARK: - MessageBubble

struct MessageBubble: View {
let message: ChatMessage
let persona: UserPersona

var body: some View {
HStack(alignment: .bottom, spacing: 8) {
```

```
if message.role == .user { Spacer(minLength: 60) }

if message.role == .assistant {
BearLogoView(size: 28)
.padding(.bottom, 4)
}

VStack(alignment: message.role == .user ? .trailing : .leading, spacing: 4) {
Text(message.text.isEmpty && message.isStreaming ? "..." : message.text)
.font(OCFont.body)
.foregroundColor(message.role == .user ? Color.OC.background : Color.OC.primaryText)
.padding(.horizontal, 14)
.padding(.vertical, 10)
.background(bubbleBackground)
.clipShape(BubbleShape(isUser: message.role == .user))

Text(timeString(message.timestamp))
.font(OCFont.caption)
.foregroundColor(Color.OC.secondaryText)
.padding(.horizontal, 4)
}

if message.role == .assistant { Spacer(minLength: 60) }
}
}

@ViewBuilder
private var bubbleBackground: some View {
if message.role == .user {
Color.OC.primary
} else {
Color.OC.surface
}
}

private func timeString(_ date: Date) -> String {
let f = DateFormatter()
f.timeStyle = .short
return f.string(from: date)
}
}

// MARK: - BubbleShape

struct BubbleShape: Shape {
let isUser: Bool
let r: CGFloat = 18

func path(in rect: CGRect) -> Path {
var p = Path()
let tl = isUser ? r : 4
let tr = isUser ? 4 : r
let bl = r
let br = r

p.move(to: CGPoint(x: rect.minX + tl, y: rect.minY))
p.addLine(to: CGPoint(x: rect.maxX - tr, y: rect.minY))
p.addArc(center: CGPoint(x: rect.maxX - tr, y: rect.minY + tr),
radius: tr, startAngle: .degrees(-90), endAngle: .degrees(0), clockwise: false)
p.addLine(to: CGPoint(x: rect.maxX, y: rect.maxY - br))
p.addArc(center: CGPoint(x: rect.maxX - br, y: rect.maxY - br),
radius: br, startAngle: .degrees(0), endAngle: .degrees(90), clockwise: false)
p.addLine(to: CGPoint(x: rect.minX + bl, y: rect.maxY))
p.addArc(center: CGPoint(x: rect.minX + bl, y: rect.maxY - bl),
radius: bl, startAngle: .degrees(90), endAngle: .degrees(180), clockwise: false)
p.addLine(to: CGPoint(x: rect.minX, y: rect.minY + tl))
p.addArc(center: CGPoint(x: rect.minX + tl, y: rect.minY + tl),
radius: tl, startAngle: .degrees(180), endAngle: .degrees(270), clockwise: false)
p.closeSubpath()
return p
}
}

// MARK: - TypingIndicator

struct TypingIndicator: View {
let name: String
```

```
@State private var phase = 0

var body: some View {
HStack(alignment: .bottom, spacing: 8) {
BearLogoView(size: 28).padding(.bottom, 4)
HStack(spacing: 5) {
ForEach(0..<3, id: \.self) { i in
Circle()
.fill(Color.OC.secondaryText)
.frame(width: 7, height: 7)
.scaleEffect(phase == i ? 1.3 : 0.85)
.animation(
.easeInOut(duration: 0.4)
.repeatForever()
.delay(Double(i) * 0.15),
value: phase
)
}
}
.padding(.horizontal, 14)
.padding(.vertical, 12)
.background(Color.OC.surface)
.clipShape(Capsule())
Spacer(minLength: 60)
}
.onAppear {
withAnimation { phase = 1 }
}
}
}

// MARK: - AffirmationBanner

struct AffirmationBanner: View {
let text: String
let onDismiss: () -> Void

var body: some View {
HStack(spacing: 12) {
Image(systemName: "heart.fill")
.foregroundColor(Color.OC.accent)
.font(.system(size: 16))
Text(text)
.font(OCFont.footnote)
.foregroundColor(Color.OC.primaryText)
.lineLimit(2)
Spacer()
Button(action: onDismiss) {
Image(systemName: "xmark")
.foregroundColor(Color.OC.secondaryText)
.font(.system(size: 12, weight: .semibold))
}
}
.padding(.horizontal, 16)
.padding(.vertical, 10)
.background(
LinearGradient(
colors: [Color.OC.accent.opacity(0.18), Color.OC.surface],
startPoint: .leading, endPoint: .trailing
)
)
.overlay(
Rectangle()
.frame(width: 3)
.foregroundColor(Color.OC.accent),
alignment: .leading
)
}
}

// MARK: - SuggestionChipsView

struct SuggestionChipsView: View {
let suggestions: [String]
let onTap: (String) -> Void

var body: some View {
```

```
ScrollView(.horizontal, showsIndicators: false) {
HStack(spacing: 8) {
ForEach(suggestions, id: \.self) { chip in
Button {
onTap(chip)
} label: {
Text(chip)
.font(OCFont.caption)
.foregroundColor(Color.OC.primary)
.padding(.horizontal, 12)
.padding(.vertical, 6)
.background(Color.OC.primary.opacity(0.12))
.clipShape(Capsule())
.overlay(
Capsule()
.strokeBorder(Color.OC.primary.opacity(0.3), lineWidth: 1)
)
}
}
}
.padding(.horizontal, 16)
.padding(.vertical, 8)
}
}
}

// MARK: - QuickActionsBar

struct QuickActionsBar: View {
let actions: [(title: String, icon: String, action: () -> Void)]

var body: some View {
ScrollView(.horizontal, showsIndicators: false) {
HStack(spacing: 10) {
ForEach(actions.indices, id: \.self) { i in
let action = actions[i]
Button {
action.action()
} label: {
HStack(spacing: 6) {
Image(systemName: action.icon)
.font(.system(size: 13))
Text(action.title)
.font(OCFont.caption)
}
.foregroundColor(Color.OC.secondaryText)
.padding(.horizontal, 12)
.padding(.vertical, 7)
.background(Color.OC.surface)
.clipShape(Capsule())
.overlay(
Capsule()
.strokeBorder(Color.OC.border, lineWidth: 1)
)
}
}
}
.padding(.horizontal, 16)
.padding(.vertical, 6)
}
.background(Color.OC.background)
}
}

// MARK: - InputBar

struct InputBar: View {
@Binding var text: String
let onSend: () -> Void
@FocusState private var focused: Bool

var body: some View {
HStack(spacing: 12) {
ZStack(alignment: .leading) {
if text.isEmpty {
Text("Message \(Image(systemName: "pawprint.fill"))…")
.font(OCFont.body)
```

```
.foregroundColor(Color.OC.secondaryText.opacity(0.6))
.padding(.horizontal, 14)
}
TextField("", text: $text, axis: .vertical)
.font(OCFont.body)
.foregroundColor(Color.OC.primaryText)
.lineLimit(1...5)
.padding(.horizontal, 14)
.focused($focused)
.submitLabel(.send)
.onSubmit {
if !text.trimmingCharacters(in: .whitespacesAndNewlines).isEmpty {
onSend()
}
}
}
.frame(minHeight: 44)
.background(Color.OC.surface)
.clipShape(RoundedRectangle(cornerRadius: 22))
.overlay(
RoundedRectangle(cornerRadius: 22)
.strokeBorder(focused ? Color.OC.primary.opacity(0.5) : Color.OC.border, lineWidth: 1)
)

Button(action: onSend) {
Image(systemName: "arrow.up")
.font(.system(size: 15, weight: .bold))
.foregroundColor(text.trimmingCharacters(in: .whitespacesAndNewlines).isEmpty
? Color.OC.secondaryText : Color.OC.background)
.frame(width: 40, height: 40)
.background(
text.trimmingCharacters(in: .whitespacesAndNewlines).isEmpty
? Color.OC.surface
: Color.OC.primary
)
.clipShape(Circle())
}
.disabled(text.trimmingCharacters(in: .whitespacesAndNewlines).isEmpty)
.animation(.easeInOut(duration: 0.15), value: text)
}
.padding(.horizontal, 12)
.padding(.vertical, 10)
.background(Color.OC.background)
}
}

// MARK: - SettingsView (inline stub — full version in SettingsView.swift)

struct SettingsView: View {
@ObservedObject var persona: UserPersona
@Environment(\.dismiss) private var dismiss

var body: some View {
NavigationStack {
List {
// Profile
Section("Profile") {
HStack {
Text("Your Name")
.foregroundColor(Color.OC.primaryText)
Spacer()
Text(persona.userName.isEmpty ? "Not set" : persona.userName)
.foregroundColor(Color.OC.secondaryText)
}
HStack {
Text("Assistant Name")
.foregroundColor(Color.OC.primaryText)
Spacer()
Text(persona.assistantName.isEmpty ? "Claw" : persona.assistantName)
.foregroundColor(Color.OC.secondaryText)
}
}

// Communication style
Section("Communication Style") {
ForEach(CommunicationStyle.allCases) { style in
HStack {
```

```
Text(style.rawValue.capitalized)
.foregroundColor(Color.OC.primaryText)
Spacer()
if persona.style == style {
Image(systemName: "checkmark")
.foregroundColor(Color.OC.primary)
}
}
.contentShape(Rectangle())
.onTapGesture { persona.style = style; persona.save() }
}
}

// Interests
Section("Interests (\(persona.interests.count))") {
ForEach(persona.interests) { interest in
HStack {
Text(interest.emoji)
Text(interest.label)
.foregroundColor(Color.OC.primaryText)
Spacer()
Toggle("", isOn: Binding(
get: { interest.notificationsEnabled },
set: { val in
if let idx = persona.interests.firstIndex(where: { $0.id == interest.id }) {
persona.interests[idx].notificationsEnabled = val
persona.save()
Task {
await HermesInterestEngine.shared
.scheduleInterestNotifications(for: persona)
}
}
}
))
.labelsHidden()
.tint(Color.OC.primary)
}
}
if persona.interests.isEmpty {
Text("No interests yet — chat to add some!")
.foregroundColor(Color.OC.secondaryText)
.font(OCFont.footnote)
}
}

// Affirmations
Section("Daily Affirmation") {
Toggle("Enabled", isOn: $persona.dailyAffirmationsEnabled)
.tint(Color.OC.primary)
.onChange(of: persona.dailyAffirmationsEnabled) { _ in
persona.save()
Task {
await HermesPersonality.shared.scheduleDailyAffirmation(for: persona)
}
}
if persona.dailyAffirmationsEnabled {
DatePicker("Time", selection: $persona.affirmationTime, displayedComponents: .hourAndMinute)
.foregroundColor(Color.OC.primaryText)
.onChange(of: persona.affirmationTime) { _ in
persona.save()
Task {
await HermesPersonality.shared.scheduleDailyAffirmation(for: persona)
}
}
}
}

// About
Section("About") {
HStack {
Text("Version")
.foregroundColor(Color.OC.primaryText)
Spacer()
Text("1.0.0")
.foregroundColor(Color.OC.secondaryText)
}
HStack {
```

```
Text("Memory entries")
.foregroundColor(Color.OC.primaryText)
Spacer()
MemoryCountBadge()
}
}
}
.scrollContentBackground(.hidden)
.background(Color.OC.background)
.listRowBackground(Color.OC.surface)
.navigationTitle("Settings")
.navigationBarTitleDisplayMode(.inline)
.toolbar {
ToolbarItem(placement: .navigationBarTrailing) {
Button("Done") { dismiss() }
.foregroundColor(Color.OC.primary)
}
}
}
.preferredColorScheme(.dark)
}
}

// MARK: - MemoryCountBadge

struct MemoryCountBadge: View {
@State private var count = 0

var body: some View {
Text("\(count)")
.foregroundColor(Color.OC.secondaryText)
.task {
let entries = await HermesMemory.shared.allEntries()
count = entries.count
}
}
}

// MARK: - Kairos shorthand

private typealias Kairos = HermesKairos
```

◆ HermesAgentHarness.swift (320 lines)

```swift
import Foundation

// MARK: - Agent Role System
//
// Each role has sharply-defined behavioural constraints and a restricted
// tool pool. Specialisation prevents role-bleed where an exploration
// agent accidentally overwrites memory, or an execution agent skips
// planning.

enum AgentRole: String, Codable, CaseIterable {
case explore // read-only investigation; never writes
case plan // synthesises findings into a step list; no side effects
case execute // carries out the plan; only role with write access
case verify // confirms execute's output; read-only, high scrutiny

var allowedTier: PermissionTier {
switch self {
case .explore: return .readonly
case .plan: return .readonly
case .execute: return .privileged
case .verify: return .readonly
}
}

var description: String {
switch self {
case .explore: return "Read-only investigation of memory and context."
case .plan: return "Synthesise findings into an ordered step list."
case .execute: return "Carry out the plan with write access."
case .verify: return "Confirm execution results and flag discrepancies."
}
}
}

// MARK: - Typed Event Stream
//
// Every harness operation emits a typed AgentEvent. These events are the
// single source of truth for logging, debugging, and dual-level verification.
//
// Bug fix #2: `Error` does not conform to `Sendable`, so `AgentEvent` must
// be `@unchecked Sendable` to cross actor boundaries under Swift 6 strict
// concurrency. We own all construction sites, so this is safe.

enum AgentEvent: @unchecked Sendable {
case sessionStarted(conversationId: String, role: AgentRole)
case toolSelected(toolId: String, role: AgentRole)
case toolValidated(toolId: String, result: HermesToolRegistry.PermissionResult)
case toolExecuted(toolId: String, success: Bool, durationMs: Int)
case budgetChecked(remaining: Int, estimatedCost: Int, allowed: Bool)
case transcriptCompacted(beforeTokens: Int, afterTokens: Int)
case workflowStepCompleted(stepName: String, index: Int, total: Int)
case verificationPassed(checkName: String)
case verificationFailed(checkName: String, reason: String)
case harnessError(Error)
}

// MARK: - Transcript Message

struct TranscriptMessage: Codable {
enum Role: String, Codable { case user, assistant, system }
let role: Role
let content: String
let timestamp: Date
var tokenEstimate: Int // rough: content.count / 4
var isPinned: Bool = false // pinned messages survive compaction

init(role: Role, content: String, timestamp: Date = Date()) {
self.role = role
self.content = content
self.timestamp = timestamp
self.tokenEstimate = max(1, content.count / 4)
}
}

// MARK: - HermesAgentHarness
```

```swift

/// Ties all subsystems together. Entry point for all agentic operations.
/// Use `run(role:task:)` to dispatch a scoped, permission-bounded agent.
actor HermesAgentHarness {
static let shared = HermesAgentHarness()

private let memory = HermesMemory.shared
private let registry = HermesToolRegistry.shared
private let session = HermesSessionState.shared
private let kairos = HermesKairos.shared
private let context = HermesContextTracker.shared

private var eventLog: [AgentEvent] = []
private var transcript: [TranscriptMessage] = []

// Token ceiling before auto-compaction triggers
private let compactionThreshold = 6_000

private init() {}

// MARK: - Agent dispatch

/// Run a scoped agent. Returns a structured result and emits events throughout.
func run(role: AgentRole, task: String) async throws -> AgentResult {
let conversationId = await session.conversation.id

emit(.sessionStarted(conversationId: conversationId, role: role))

// Pre-turn token budget check
let budget = await session.tokenBudget
let allowed = budget.canStartTurn(estimatedCost: 500)
emit(.budgetChecked(remaining: budget.remaining, estimatedCost: 500, allowed: allowed))
guard allowed else { throw SessionError.tokenBudgetExhausted(used: budget.used, limit: budget.sessionLimit) }

// Build context-appropriate tool pool
let contextTags = Set((await context.currentTopic() ?? "").components(separatedBy: ", "))
let pool = await registry.assemblePool(for: role, contextTags: contextTags)

// Record the task in the transcript
append(message: TranscriptMessage(role: .user, content: task))

// Auto-compact if approaching threshold
if transcriptTokenCount() > compactionThreshold {
await compactTranscript()
}

// Build permission context for this agent
let permCtx = PermissionContext(
sessionTier: role.allowedTier,
isInForeground: true,
tokenBudgetRemaining: budget.remaining,
kairosActive: false,
metadata: ["agentRole": role.rawValue]
)

// Execute tools in pool order (explore/plan are read-only; execute writes)
var toolResults: [String: Any] = [:]
for tool in pool {
emit(.toolSelected(toolId: tool.id, role: role))

let validation = await registry.validate(tool: tool.id, context: permCtx)
emit(.toolValidated(toolId: tool.id, result: validation))
guard validation.isAllowed else { continue }

let start = Date()
let result = await executeTool(tool, context: permCtx)
let ms = Int(Date().timeIntervalSince(start) * 1000)
emit(.toolExecuted(toolId: tool.id, success: result.success, durationMs: ms))

if result.success { toolResults[tool.id] = result.value }
}

// Build a response via the LLM (Hermes context injected inside HermesLLMClient)
let llmMessages = transcript.map {
LLMMessage(role: $0.role == .user ? .user : .assistant, content: $0.content)
}
// Pass a minimal base prompt — HermesLLMClient.buildSystemPrompt()
```

```
// enriches it with role instructions + full Hermes context automatically.
let llmRequest = LLMRequest(
systemPrompt: "You are AppClaw, a helpful and proactive AI assistant.",
messages: llmMessages,
tools: pool,
maxTokens: min(4096, budget.remaining),
role: role
)
let llmResponse = try await HermesLLMClient.shared.complete(llmRequest)
let response = llmResponse.content
append(message: TranscriptMessage(role: .assistant, content: response))

// Dual-level verification
let agentVerification = verifyAgentOutput(response: response, role: role)
let harnessVerification = verifyHarness(toolResults: toolResults, role: role)

for check in agentVerification + harnessVerification {
emit(check.passed
? .verificationPassed(checkName: check.name)
: .verificationFailed(checkName: check.name, reason: check.reason ?? ""))
}

// Record token usage (rough estimate)
let promptEst = transcriptTokenCount()
let replyEst = max(1, response.count / 4)
try await session.recordTokenUsage(prompt: promptEst, completion: replyEst)

return AgentResult(
role: role,
response: response,
toolsUsed: Array(toolResults.keys),
verificationPassed: (agentVerification + harnessVerification).allSatisfy(\.passed),
events: eventLog
)
}

// MARK: - Tool execution (stub — wire up real tool handlers here)

private func executeTool(_ tool: ToolDefinition,
context: PermissionContext) async -> ToolResult {
switch tool.id {
case "hermes.memory.search":
return ToolResult(success: true, value: "memory_results_placeholder")
case "hermes.context.topic":
let topic = await self.context.currentTopic() ?? "unknown"
return ToolResult(success: true, value: topic)
case "hermes.dream.trigger":
await HermesDreamEngine.shared.runDreamCycle()
return ToolResult(success: true, value: "dream_complete")
case "hermes.suggestions.refresh":
await HermesProactiveEngine.shared.refresh()
return ToolResult(success: true, value: "suggestions_refreshed")
default:
return ToolResult(success: false, value: nil)
}
}

// MARK: - Transcript compaction
//
// Keeps pinned messages and the most recent tail intact.
// Summarises older messages into a single system-role placeholder.
// This prevents context entropy while staying within token limits.

private func compactTranscript() async {
let before = transcriptTokenCount()
let pinned = transcript.filter(\.isPinned)
let unpinned = transcript.filter { !$0.isPinned }

// Keep last 10 unpinned messages; summarise the rest
let keep = Array(unpinned.suffix(10))
let summarised = unpinned.dropLast(10)

if summarised.isEmpty { return }

let summary = "[Compacted \(summarised.count) earlier messages. " +
"Topics covered: \(topicSummary(of: summarised))]"
let placeholder = TranscriptMessage(role: .system, content: summary)
```

```
transcript = pinned + [placeholder] + keep
let after = transcriptTokenCount()
emit(.transcriptCompacted(beforeTokens: before, afterTokens: after))
}

private func topicSummary(of messages: [TranscriptMessage]) -> String {
let words = messages.map(\.content).joined(separator: " ")
.lowercased()
.components(separatedBy: .alphanumerics.inverted)
.filter { $0.count > 4 }
let freq = Dictionary(words.map { ($0, 1) }, uniquingKeysWith: +)
return freq.sorted { $0.value > $1.value }.prefix(5).map(\.key).joined(separator: ", ")
}

private func transcriptTokenCount() -> Int {
transcript.reduce(0) { $0 + $1.tokenEstimate }
}

private func append(message: TranscriptMessage) {
transcript.append(message)
}

// MARK: - Dual-level verification

private struct VerificationCheck {
let name: String
let passed: Bool
let reason: String?
}

/// Level 1 — verify the agent's own output (content checks).
private func verifyAgentOutput(response: String, role: AgentRole) -> [VerificationCheck] {
[
VerificationCheck(
name: "response_non_empty",
passed: !response.isEmpty,
reason: response.isEmpty ? "Agent produced an empty response." : nil
),
VerificationCheck(
name: "no_hardcoded_secrets",
passed: !response.lowercased().contains("api_key") &&
!response.lowercased().contains("password"),
reason: "Response may contain sensitive literals."
),
]
}

/// Level 2 — verify the harness (structural / permission checks).
private func verifyHarness(toolResults: [String: Any], role: AgentRole) -> [VerificationCheck] {
let writeToolsUsed = toolResults.keys.filter {
$0.contains("write") || $0.contains("observe") || $0.contains("dream")
}
return [
VerificationCheck(
name: "no_writes_in_readonly_role",
passed: role != .explore || writeToolsUsed.isEmpty,
reason: writeToolsUsed.isEmpty ? nil
: "Explore role invoked write tool(s): \(writeToolsUsed.joined(separator: ", "))"
),
]
}

// MARK: - Event log

private func emit(_ event: AgentEvent) {
eventLog.append(event)
#if DEBUG
print("[Harness] \(event)")
#endif
}

func recentEvents(limit: Int = 50) -> [AgentEvent] {
Array(eventLog.suffix(limit))
}
}
```

```
// MARK: - Supporting types

struct ToolResult {
let success: Bool
let value: Any?
}

struct AgentResult {
let role: AgentRole
let response: String
let toolsUsed: [String]
let verificationPassed: Bool
let events: [AgentEvent]
}
```

◆ HermesAutomation.swift (285 lines)

```swift
import Foundation
import UIKit

// MARK: - HermesAutomation
//
// On-device automation within Apple guidelines:
// • Spending / habit tracker — user logs manually; AI summarises
// • App launcher — opens installed apps via URL schemes
// • Task memory — "remember to reorder X" stored & surfaced
// • Quick actions — reorder Starbucks, open Maps, etc.
//
// Anything requiring cross-app data access uses the Shortcuts app
// (the user creates the Shortcut; we deep-link into it).

// MARK: - SpendingEntry

struct SpendingEntry: Codable, Identifiable {
var id: UUID = UUID()
var amount: Double
var category: SpendingCategory
var merchant: String
var note: String?
var date: Date = Date()

enum SpendingCategory: String, Codable, CaseIterable, Identifiable {
case fastFood = "Fast Food"
case coffee = "Coffee"
case groceries = "Groceries"
case entertainment = "Entertainment"
case transport = "Transport"
case health = "Health"
case shopping = "Shopping"
case other = "Other"

var id: String { rawValue }
var emoji: String {
switch self {
case .fastFood: return "■"
case .coffee: return "■■"
case .groceries: return "■"
case .entertainment: return "■"
case .transport: return "■"
case .health: return "■"
case .shopping: return "■"
case .other: return "■"
}
}
}
}

// MARK: - SavedTask (things like "reorder Starbucks")

struct SavedTask: Codable, Identifiable {
var id: UUID = UUID()
var title: String // "Reorder my Starbucks drink"
var action: TaskAction
var createdAt: Date = Date()
var lastTriggered: Date?

enum TaskAction: Codable {
case openURL(String) // URL scheme / universal link
case openShortcut(String) // Shortcuts app deep-link
case sendNotification(String) // just remind the user
case customPrompt(String) // ask the LLM to help
}
}

// MARK: - HermesAutomation actor

actor HermesAutomation {
static let shared = HermesAutomation()

private var spendingLog: [SpendingEntry] = []
private var savedTasks: [SavedTask] = []

```

```
private let spendingURL: URL = fileURL("spending_log.json")
private let tasksURL: URL = fileURL("saved_tasks.json")

private let encoder: JSONEncoder = { let e = JSONEncoder(); e.dateEncodingStrategy = .iso8601; e.outputFormatting…
private let decoder: JSONDecoder = { let d = JSONDecoder(); d.dateDecodingStrategy = .iso8601; return d }()

private init() {
Task { await load() }
}

// MARK: - Spending tracker

func logSpending(amount: Double, category: SpendingEntry.SpendingCategory,
merchant: String, note: String? = nil) async {
let entry = SpendingEntry(amount: amount, category: category,
merchant: merchant, note: note)
spendingLog.append(entry)
saveSpending()

// Also write to Hermes memory so AI can reference it
try? await HermesMemory.shared.observe(
category: "spending",
content: ["amount": amount, "category": category.rawValue, "merchant": merchant],
metadata: ["importance": 2]
)
}

/// Summary for the LLM to reference — totals by category for the current month.
func monthlySummary() -> String {
let cal = Calendar.current
let now = Date()
let month = cal.component(.month, from: now)
let year = cal.component(.year, from: now)

let thisMonth = spendingLog.filter {
cal.component(.month, from: $0.date) == month &&
cal.component(.year, from: $0.date) == year
}

guard !thisMonth.isEmpty else { return "No spending logged this month." }

let byCategory = Dictionary(grouping: thisMonth, by: \.category)
let lines = byCategory.map { cat, entries -> String in
let total = entries.reduce(0) { $0 + $1.amount }
return "\(cat.emoji) \(cat.rawValue): $\(String(format: "%.2f", total))"
}.sorted()

let grandTotal = thisMonth.reduce(0) { $0 + $1.amount }
return "This month:\n" + lines.joined(separator: "\n") +
"\n\nTotal: $\(String(format: "%.2f", grandTotal))"
}

func recentEntries(limit: Int = 10) -> [SpendingEntry] {
Array(spendingLog.suffix(limit).reversed())
}

// MARK: - Saved tasks

func saveTask(_ task: SavedTask) async {
savedTasks.removeAll { $0.title.lowercased() == task.title.lowercased() }
savedTasks.append(task)
saveTasks()
}

func allTasks() -> [SavedTask] { savedTasks }

/// Execute a saved task — opens the appropriate app or runs the action.
@MainActor
func execute(task: SavedTask) async {
var updated = task
updated.lastTriggered = Date()
await HermesAutomation.shared._updateTask(updated)

switch task.action {
case .openURL(let urlString):
if let url = URL(string: urlString), UIApplication.shared.canOpenURL(url) {
await UIApplication.shared.open(url)
```

```
}
case .openShortcut(let name):
let encoded = name.addingPercentEncoding(withAllowedCharacters: .urlQueryAllowed) ?? name
if let url = URL(string: "shortcuts://run-shortcut?name=\(encoded)") {
await UIApplication.shared.open(url)
}
case .sendNotification(let body):
let content = UNMutableNotificationContent()
content.title = "■ Reminder"
content.body = body
content.sound = .default
let req = UNNotificationRequest(
identifier: task.id.uuidString,
content: content,
trigger: UNTimeIntervalNotificationTrigger(timeInterval: 1, repeats: false)
)
try? await UNUserNotificationCenter.current().add(req)
case .customPrompt:
break // handled in ChatViewModel
}
}

func _updateTask(_ task: SavedTask) {
if let idx = savedTasks.firstIndex(where: { $0.id == task.id }) {
savedTasks[idx] = task
saveTasks()
}
}

// MARK: - Natural language task parser

/// Detects automation intent from a user message.
/// Returns a SavedTask if one can be created from the text.
func detectTask(from text: String) -> SavedTask? {
let lower = text.lowercased()

// Starbucks reorder
if lower.contains("starbucks") && (lower.contains("reorder") || lower.contains("order") || lower.contains("my…
return SavedTask(
title: "Reorder my Starbucks",
action: .openURL("starbucks://")
)
}
// Uber / Lyft
if lower.contains("uber") || lower.contains("lyft") {
let scheme = lower.contains("lyft") ? "lyft://" : "uber://"
return SavedTask(title: "Open ride share", action: .openURL(scheme))
}
// DoorDash / Uber Eats
if lower.contains("doordash") {
return SavedTask(title: "Open DoorDash", action: .openURL("doordash://"))
}
if lower.contains("uber eats") {
return SavedTask(title: "Open Uber Eats", action: .openURL("ubereats://"))
}
// Maps / navigation
if lower.contains("navigate") || lower.contains("directions") {
return SavedTask(title: "Open Maps", action: .openURL("maps://"))
}
// Spending tracker
if lower.contains("track") && (lower.contains("spend") || lower.contains("spending") || lower.contains("money…
return SavedTask(
title: "Log spending",
action: .customPrompt("Help me log a spending entry")
)
}
// Generic "remind me"
if lower.contains("remind me") {
return SavedTask(
title: String(text.prefix(60)),
action: .sendNotification(text)
)
}
return nil
}

// MARK: - App URL schemes catalogue
```

```
static let appSchemes: [(name: String, emoji: String, scheme: String)] = [
("Starbucks", "■■", "starbucks://"),
("Uber", "■", "uber://"),
("Lyft", "■", "lyft://"),
("DoorDash", "■", "doordash://"),
("Uber Eats", "■", "ubereats://"),
("Maps", "■", "maps://"),
("Spotify", "■", "spotify://"),
("Netflix", "■", "nflx://"),
("Instagram", "■", "instagram://"),
("Twitter/X", "■", "twitter://"),
("YouTube", "■■", "youtube://"),
("Amazon", "■", "amazon://"),
("PayPal", "■", "paypal://"),
("Venmo", "■", "venmo://"),
("Snapchat", "■", "snapchat://"),
("Gmail", "■", "googlegmail://"),
]

// MARK: - Persistence

private func load() async {
if let data = try? Data(contentsOf: spendingURL),
let decoded = try? decoder.decode([SpendingEntry].self, from: data) {
spendingLog = decoded
}
if let data = try? Data(contentsOf: tasksURL),
let decoded = try? decoder.decode([SavedTask].self, from: data) {
savedTasks = decoded
}
}

private func saveSpending() {
if let data = try? encoder.encode(spendingLog) {
try? data.write(to: spendingURL, options: .atomic)
}
}

private func saveTasks() {
if let data = try? encoder.encode(savedTasks) {
try? data.write(to: tasksURL, options: .atomic)
}
}

private static func fileURL(_ name: String) -> URL {
let dir = FileManager.default.urls(for: .documentDirectory, in: .userDomainMask)[0]
.appendingPathComponent("hermes")
try? FileManager.default.createDirectory(at: dir, withIntermediateDirectories: true)
return dir.appendingPathComponent(name)
}
}

// MARK: - UNUserNotificationCenter async shim

import UserNotifications
```

◆ HermesContextTracker.swift (139 lines)

```swift
import Foundation

// MARK: - Role

enum MessageRole {
case user
case assistant
}

// MARK: - TrackedMessage

private struct TrackedMessage {
let role: MessageRole
let text: String
let timestamp: Date
}

// MARK: - HermesContextTracker
//
// Maintains a sliding window of recent messages and extracts the current
// topic + intent without any network calls.
//
// Used by HermesIntegration.currentTopic() and HermesProactiveEngine
// for richer, more accurate suggestions.

actor HermesContextTracker {
static let shared = HermesContextTracker()

/// How many messages to keep in the active window.
private let windowSize = 20

private var window: [TrackedMessage] = []

// Cache so we don't re-derive on every call
private var cachedTopic: String?
private var topicDirty = true

private init() {}

// MARK: - Ingestion

/// Add a new message to the sliding window.
func ingest(text: String, role: MessageRole) {
let msg = TrackedMessage(role: role, text: text, timestamp: Date())
window.append(msg)
if window.count > windowSize { window.removeFirst() }
topicDirty = true
}

// MARK: - Derived context

/// Best-guess "current topic" from the recent window.
/// Returns nil if the window is empty.
func currentTopic() -> String? {
guard !window.isEmpty else { return nil }
if !topicDirty, let cached = cachedTopic { return cached }
let topic = deriveTopic()
cachedTopic = topic
topicDirty = false
return topic
}

/// Rough intent category of the most recent user message.
func currentIntent() -> Intent? {
guard let lastUser = window.last(where: { $0.role == .user }) else { return nil }
return classifyIntent(lastUser.text)
}

/// Plain summary of the window — useful for injecting context into a new conversation.
func sessionSummary() -> String {
let userLines = window
.filter { $0.role == .user }
.suffix(5)
.map { "• \($0.text.prefix(120))" }
.joined(separator: "\n")
```

```
guard !userLines.isEmpty else { return "No activity yet." }
return "Recent messages:\n\(userLines)"
}

// MARK: - Topic derivation

private static let stopwords: Set<String> = [
"the","and","for","are","but","not","you","all","can","had","her","was","one",
"our","out","day","get","has","him","his","how","man","new","now","old","see",
"two","way","who","did","its","let","put","say","she","too","use","that","with",
"have","this","will","your","from","they","know","want","been","good","much",
"some","time","very","when","come","here","just","like","long","make","many",
"more","only","over","such","take","than","them","well","were","what","also",
"into","most","then","there","these","think","those","about","after","being",
"could","going","their","where","which","would","should","because"
]

private func deriveTopic() -> String? {
// Weight user messages 2× over assistant messages
let corpus = window.map { msg -> String in
let weight = msg.role == .user ? 2 : 1
return Array(repeating: msg.text, count: weight).joined(separator: " ")
}.joined(separator: " ")

let words = corpus
.lowercased()
.components(separatedBy: .alphanumerics.inverted)
.filter { $0.count > 3 && !Self.stopwords.contains($0) }

let freq = Dictionary(words.map { ($0, 1) }, uniquingKeysWith: +)
let top = freq.sorted { $0.value > $1.value }.prefix(3).map(\.key)
return top.isEmpty ? nil : top.joined(separator: ", ")
}

// MARK: - Intent classification

enum Intent: String {
case question // asking for information
case debugRequest // fixing a bug or error
case buildRequest // creating something new
case explainRequest // asking for an explanation
case editRequest // modifying existing content
case other
}

private func classifyIntent(_ text: String) -> Intent {
let t = text.lowercased()

let debugSignals = ["error","bug","fix","crash","fail","broken","not working","exception","issue"]
let buildSignals = ["create","build","make","add","implement","generate","write","new"]
let explainSignals = ["explain","what is","what are","how does","why does","tell me","describe"]
let editSignals = ["change","update","modify","rename","refactor","move","delete","remove"]
let questionSignals = ["?","how","what","when","where","who","which","why","can you"]

func matches(_ signals: [String]) -> Bool { signals.contains { t.contains($0) } }

if matches(debugSignals) { return .debugRequest }
if matches(buildSignals) { return .buildRequest }
if matches(explainSignals) { return .explainRequest }
if matches(editSignals) { return .editRequest }
if matches(questionSignals){ return .question }
return .other
}
}
```

◆ HermesCronScheduler.swift (262 lines)

```swift
import Foundation
import UserNotifications

// MARK: - CronJob model

struct CronJob: Codable, Identifiable {
var id: UUID = UUID()
var title: String // "Remind me to drink water"
var body: String // full notification text
var schedule: JobSchedule
var isEnabled: Bool = true
var lastFired: Date?
var createdAt: Date = Date()
var category: JobCategory = .reminder

enum JobCategory: String, Codable, CaseIterable, Identifiable {
case reminder, habit, checkin, task
var id: String { rawValue }
var emoji: String {
switch self {
case .reminder: return "■"
case .habit: return "■"
case .checkin: return "■"
case .task: return "■"
}
}
}

// MARK: - Schedule types

enum JobSchedule: Codable, Equatable {
case daily(hour: Int, minute: Int)
case weekly(weekday: Int, hour: Int, minute: Int) // 1=Sun … 7=Sat
case interval(minutes: Int) // every N minutes (min 15 for background)
case once(date: Date)

var humanReadable: String {
switch self {
case .daily(let h, let m):
return "Every day at \(formatted(h, m))"
case .weekly(let wd, let h, let m):
let days = ["Sun","Mon","Tue","Wed","Thu","Fri","Sat"]
let day = days[max(0, min(wd - 1, 6))]
return "Every \(day) at \(formatted(h, m))"
case .interval(let mins):
return mins >= 60 ? "Every \(mins / 60)h" : "Every \(mins) min"
case .once(let date):
let f = DateFormatter()
f.dateStyle = .short; f.timeStyle = .short
return "Once — \(f.string(from: date))"
}
}

private func formatted(_ h: Int, _ m: Int) -> String {
let ampm = h >= 12 ? "PM" : "AM"
let h12 = h == 0 ? 12 : (h > 12 ? h - 12 : h)
return String(format: "%d:%02d %@", h12, m, ampm)
}
}

// MARK: - HermesCronScheduler

/// Manages user-defined scheduled jobs using UNUserNotificationCenter.
/// No background daemon required — iOS delivers the notifications at the
/// scheduled time even if the app is closed. When the user taps a
/// notification, the app opens and the job's payload is available.
actor HermesCronScheduler {
static let shared = HermesCronScheduler()

private var jobs: [CronJob] = []
private let saveURL: URL = {
FileManager.default.urls(for: .documentDirectory, in: .userDomainMask)[0]
.appendingPathComponent("hermes/cron_jobs.json")
}()
```

```

private let encoder: JSONEncoder = {
let e = JSONEncoder(); e.dateEncodingStrategy = .iso8601; return e
}()
private let decoder: JSONDecoder = {
let d = JSONDecoder(); d.dateDecodingStrategy = .iso8601; return d
}()

private init() {
Task { await load() }
}

// MARK: - CRUD

/// Add a new job and schedule its notification.
func add(_ job: CronJob) async {
jobs.append(job)
save()
if job.isEnabled { schedule(job) }
}

/// Update an existing job.
func update(_ job: CronJob) async {
guard let idx = jobs.firstIndex(where: { $0.id == job.id }) else { return }
cancel(jobs[idx])
jobs[idx] = job
save()
if job.isEnabled { schedule(job) }
}

/// Delete a job and cancel its notification.
func delete(id: UUID) async {
if let job = jobs.first(where: { $0.id == id }) { cancel(job) }
jobs.removeAll { $0.id == id }
save()
}

func allJobs() -> [CronJob] { jobs }

// MARK: - Scheduling

private func schedule(_ job: CronJob) {
let content = UNMutableNotificationContent()
content.title = "■ \(job.category.emoji) \(job.title)"
content.body = job.body
content.sound = .default
content.userInfo = ["jobId": job.id.uuidString]

let trigger: UNNotificationTrigger

switch job.schedule {
case .daily(let h, let m):
var c = DateComponents(); c.hour = h; c.minute = m
trigger = UNCalendarNotificationTrigger(dateMatching: c, repeats: true)

case .weekly(let wd, let h, let m):
var c = DateComponents(); c.weekday = wd; c.hour = h; c.minute = m
trigger = UNCalendarNotificationTrigger(dateMatching: c, repeats: true)

case .interval(let mins):
let secs = TimeInterval(max(15, mins) * 60)
trigger = UNTimeIntervalNotificationTrigger(timeInterval: secs, repeats: true)

case .once(let date):
let interval = max(1, date.timeIntervalSinceNow)
trigger = UNTimeIntervalNotificationTrigger(timeInterval: interval, repeats: false)
}

let request = UNNotificationRequest(
identifier: notifId(job),
content: content,
trigger: trigger
)
UNUserNotificationCenter.current().add(request)
}

private func cancel(_ job: CronJob) {
```

```
UNUserNotificationCenter.current()
.removePendingNotificationRequests(withIdentifiers: [notifId(job)])
}

private func notifId(_ job: CronJob) -> String { "cron_\(job.id.uuidString)" }

// MARK: - Persistence

private func load() async {
guard let data = try? Data(contentsOf: saveURL),
let decoded = try? decoder.decode([CronJob].self, from: data) else { return }
jobs = decoded
}

private func save() {
try? FileManager.default.createDirectory(
at: saveURL.deletingLastPathComponent(),
withIntermediateDirectories: true
)
if let data = try? encoder.encode(jobs) {
try? data.write(to: saveURL, options: .atomic)
}
}

// MARK: - Natural language parser
// Converts phrases like "every day at 8am", "every Monday at 9am",
// "every 30 minutes" into a JobSchedule.

static func parseSchedule(from text: String) -> JobSchedule? {
let lower = text.lowercased()

// "every N minutes"
if let mins = extractNumber(after: "every", before: "minute", in: lower) {
return .interval(minutes: mins)
}
// "every hour" / "every N hours"
if lower.contains("every hour") { return .interval(minutes: 60) }
if let hrs = extractNumber(after: "every", before: "hour", in: lower) {
return .interval(minutes: hrs * 60)
}

// Parse time like "8am", "9:30pm", "14:00"
let (hour, minute) = extractTime(from: lower) ?? (9, 0)

// "every day" / "daily"
if lower.contains("every day") || lower.contains("daily") {
return .daily(hour: hour, minute: minute)
}

// "every monday" etc.
let weekdays = ["sunday":1,"monday":2,"tuesday":3,"wednesday":4,
"thursday":5,"friday":6,"saturday":7]
for (name, num) in weekdays where lower.contains(name) {
return .weekly(weekday: num, hour: hour, minute: minute)
}

// "every morning" → 8am, "every evening" → 7pm, "every night" → 9pm
if lower.contains("every morning") { return .daily(hour: 8, minute: 0) }
if lower.contains("every evening") { return .daily(hour: 19, minute: 0) }
if lower.contains("every night") { return .daily(hour: 21, minute: 0) }
if lower.contains("every lunch") { return .daily(hour: 12, minute: 0) }

// Fallback: daily at extracted time
if lower.contains("every") {
return .daily(hour: hour, minute: minute)
}

return nil
}

private static func extractNumber(after prefix: String, before suffix: String, in text: String) -> Int? {
let pattern = "\(prefix)\\s+(\\d+)\\s+\(suffix)"
guard let regex = try? NSRegularExpression(pattern: pattern),
let match = regex.firstMatch(in: text, range: NSRange(text.startIndex..., in: text)),
let range = Range(match.range(at: 1), in: text) else { return nil }
return Int(text[range])
}
```

```
230
231 private static func extractTime(from text: String) -> (Int, Int)? {
232 // "8am", "9:30pm", "14:00", "3 pm"
233 let patterns = [
234 "(\\d{1,2}):(\\d{2})\\s*(am|pm)?",
235 "(\\d{1,2})\\s*(am|pm)"
236 ]
237 for pattern in patterns {
238 guard let regex = try? NSRegularExpression(pattern: pattern, options: .caseInsensitive),
239 let match = regex.firstMatch(in: text, range: NSRange(text.startIndex..., in: text))
240 else { continue }
241
242 let hourStr = match.range(at: 1).location != NSNotFound ? substring(match.range(at: 1), in: text) : nil
243 let minStr = match.range(at: 2).location != NSNotFound && pattern.contains(":(") ? substring(match.rang…
244 let meridiem = (match.numberOfRanges > 3) ? substring(match.range(at: match.numberOfRanges - 1), in: text…
245
246 guard var hour = Int(hourStr ?? "9") else { continue }
247 let minute = Int(minStr ?? "0") ?? 0
248
249 if let m = meridiem {
250 if m.lowercased() == "pm" && hour != 12 { hour += 12 }
251 if m.lowercased() == "am" && hour == 12 { hour = 0 }
252 }
253 return (hour, minute)
254 }
255 return nil
256 }
257
258 private static func substring(_ nsRange: NSRange, in text: String) -> String? {
259 guard let range = Range(nsRange, in: text) else { return nil }
260 return String(text[range])
261 }
262 }
```

◆ HermesDreamEngine.swift (318 lines)

```swift
import Foundation
import BackgroundTasks

// MARK: - HermesDreamEngine
//
// "Dream mode" — runs while the app is backgrounded, targeting ~2 am local time.
//
// Improvements over v1:
// - Batch memory updates: all promotions in one persist call (was N separate calls)
// - Smart 2 am scheduling: aims for the next 2 am local time, not "5 hours from now"
// - Proper English stopword list for topic insight generation
// - Removed unused `dreamLog: [DreamEntry]`
// - `handleDream` no longer captures `self` strongly in nested closure
// - `phase4_selfImprove` scoped to recent 24 h, not all-time

final class HermesDreamEngine {
static let shared = HermesDreamEngine()

private let taskIdentifier = "com.openclaw.hermes.dream"
private let memory = HermesMemory.shared

private init() {}

// MARK: - Registration

func registerBackgroundTask() {
BGTaskScheduler.shared.register(forTaskWithIdentifier: taskIdentifier, using: nil) { [weak self] task in
guard let self, let processingTask = task as? BGProcessingTask else {
task.setTaskCompleted(success: false)
return
}
self.handleDream(task: processingTask)
}
}

/// Call when the app moves to background. Schedules the next dream at ~2 am local.
func scheduleNextDream() {
let request = BGProcessingTaskRequest(identifier: taskIdentifier)
request.requiresNetworkConnectivity = false
request.requiresExternalPower = false
request.earliestBeginDate = nextDreamDate()
try? BGTaskScheduler.shared.submit(request)
}

/// Next occurrence of 2 am local time. If it's already past 2 am today, uses tomorrow.
private func nextDreamDate() -> Date {
let cal = Calendar.current
var components = cal.dateComponents([.year, .month, .day], from: Date())
components.hour = 2
components.minute = 0
components.second = 0
let todayAt2 = cal.date(from: components) ?? Date()
// If we've passed today's 2 am, aim for tomorrow
return todayAt2 > Date() ? todayAt2 : todayAt2.addingTimeInterval(86400)
}

// MARK: - Dream execution

private func handleDream(task: BGProcessingTask) {
let dreamTask = Task { [weak self] in
guard let self else { return }
await self.runDreamCycle()
task.setTaskCompleted(success: true)
self.scheduleNextDream()
}
task.expirationHandler = { dreamTask.cancel() }
}

/// Full dream cycle. Also callable manually (e.g., from a Debug/Settings screen).
@discardableResult
func runDreamCycle() async -> DreamReport {
var report = DreamReport(startedAt: Date())

await phase1_consolidate(&report)
await phase2_promotePatterns(&report)
```

```
await phase3_generateInsights(&report)
await phase4_selfImprove(&report)
await phase5_resolveContradictions(&report)

report.finishedAt = Date()
await saveDreamReport(report)
return report
}

// MARK: - Phase 1: Consolidate

private func phase1_consolidate(_ report: inout DreamReport) async {
let before = await memory.allEntries().count
do {
try await memory.consolidate(importanceThreshold: 2, keepWindow: 7 * 86400)
let after = await memory.allEntries().count
report.pruned = max(0, before - after)
report.phases.append("Pruned \(report.pruned) low-importance entries.")
} catch {
report.phases.append("Consolidation error: \(error)")
}
}

// MARK: - Phase 2: Promote patterns (single batch persist)

private func phase2_promotePatterns(_ report: inout DreamReport) async {
let all = await memory.allEntries()

let freq = Dictionary(grouping: all, by: \.category).mapValues(\.count)

// Collect all mutations first, then write once
var updated: [MemoryEntry] = []
var promotedIDs: [UUID] = []

for entry in all {
let count = freq[entry.category, default: 0]
if count > 5 && entry.importance < 4 {
var e = entry
e = MemoryEntry(
id: e.id, timestamp: e.timestamp, category: e.category,
content: e.content.value,
metadata: e.metadata.mapValues(\.value),
importance: min(e.importance + 1, 5),
tier: e.tier
)
updated.append(e)
}
// Promote high-importance entries to long-term tier
if entry.importance >= 4 && entry.tier == .shortTerm {
promotedIDs.append(entry.id)
}
}

do {
if !updated.isEmpty { try await memory.updateBatch(updated) }
if !promotedIDs.isEmpty { try await memory.promoteToLongTerm(promotedIDs) }
} catch {
report.phases.append("Promotion error: \(error)")
}

report.promoted = updated.count
report.phases.append("Promoted \(updated.count) entries; moved \(promotedIDs.count) to long-term.")
}

// MARK: - Phase 3: Generate insights

private static let stopwords: Set<String> = [
"the","and","for","are","but","not","you","all","can","had","her","was","one",
"our","out","day","get","has","him","his","how","man","new","now","old","see",
"two","way","who","boy","did","its","let","put","say","she","too","use","that",
"with","have","this","will","your","from","they","know","want","been","good",
"much","some","time","very","when","come","here","just","like","long","make",
"many","more","only","over","such","take","than","them","well","were","what",
"also","into","most","other","said","then","there","these","think","those",
"about","after","being","could","every","going","great","their","where","which",
"would","should","would","because","through"
]
```

```swift

private func phase3_generateInsights(_ report: inout DreamReport) async {
let recentMessages = await memory.entries(for: "user_message")
.prefix(100)
.compactMap { ($0.content.value as? [String: Any])?["text"] as? String }

guard !recentMessages.isEmpty else {
report.phases.append("No recent messages to analyse.")
return
}

let words = recentMessages.joined(separator: " ")
.lowercased()
.components(separatedBy: .alphanumerics.inverted)
.filter { $0.count > 3 && !Self.stopwords.contains($0) }

let freq = Dictionary(words.map { ($0, 1) }, uniquingKeysWith: +)
let topWords = freq.sorted { $0.value > $1.value }.prefix(8).map(\.key)

guard !topWords.isEmpty else { return }

let insight = "Recurring topics in recent conversations: \(topWords.joined(separator: ", "))."
report.insights.append(insight)
report.phases.append("Topic insight: \(insight)")

try? await memory.observe(
category: "dream_insight",
content: ["insight": insight, "basedOn": recentMessages.count, "topWords": topWords],
metadata: ["importance": 4]
)
}

// MARK: - Phase 4: Self-improve

private func phase4_selfImprove(_ report: inout DreamReport) async {
// Scope to last 24 h only — all-time error counts are noisy
let since = Date().addingTimeInterval(-86400)
let errors = await memory.entries(for: "tool_exec")
.filter { $0.timestamp >= since }
.filter {
guard let d = $0.content.value as? [String: Any],
let ok = d["success"] as? Bool else { return false }
return !ok
}

guard errors.count >= 3 else {
report.phases.append("Self-improvement: no recurring failures in last 24 h.")
return
}

let toolNames = errors.compactMap { ($0.content.value as? [String: Any])?["tool"] as? String }
let counts = Dictionary(toolNames.map { ($0, 1) }, uniquingKeysWith: +)
let worst = counts.sorted { $0.value > $1.value }.prefix(2)
.map { "\($0.key) (\($0.value)x)" }
let note = "Heads-up: repeated failures in \(worst.joined(separator: ", ")) over the last 24 h. Consider revi…

report.improvements.append(note)
report.phases.append("Self-improvement note written.")

try? await memory.observe(
category: "self_improvement",
content: ["note": note, "errorCount": errors.count],
metadata: ["importance": 5]
)
}

// MARK: - Phase 5: Resolve contradictions
//
// Looks for entries in the same category where the content directly
// contradicts a later entry (e.g. two self_improvement notes about the
// same tool with conflicting advice, or two dream_insights with
// opposite conclusions).
//
// Strategy: when two entries in the same category share the same top
// keyword AND were written > 1 h apart, keep only the newer one and
// mark the older as superseded (importance → 1 so it gets pruned next cycle).

```

```
private func phase5_resolveContradictions(_ report: inout DreamReport) async {
let candidateCategories = ["dream_insight", "self_improvement", "kairos_insight"]
var resolved = 0

for category in candidateCategories {
let entries = await memory.entries(for: category)
guard entries.count >= 2 else { continue }

// Group by dominant keyword (first non-stopword word in content text)
var groups: [String: [MemoryEntry]] = [:]
for entry in entries {
let text = (entry.content.value as? [String: Any])?
.values.compactMap { $0 as? String }.joined(separator: " ")
?? "\(entry.content.value)"
if let key = dominantKeyword(text) {
groups[key, default: []].append(entry)
}
}

for (_, group) in groups where group.count >= 2 {
// Sort newest first; demote all but the newest
let sorted = group.sorted { $0.timestamp > $1.timestamp }
let toSupersede = sorted.dropFirst()
.filter { $0.importance > 1 }

let superseded: [MemoryEntry] = toSupersede.map { entry in
MemoryEntry(
id: entry.id, timestamp: entry.timestamp,
category: entry.category,
content: entry.content.value,
metadata: entry.metadata.mapValues(\.value),
importance: 1, // will be pruned next consolidation
tier: entry.tier
)
}

if !superseded.isEmpty {
try? await memory.updateBatch(superseded)
resolved += superseded.count
}
}
}

report.resolved = resolved
report.phases.append(
resolved > 0
? "Contradiction resolution: superseded \(resolved) outdated entries."
: "Contradiction resolution: no contradictions found."
)
}

private func dominantKeyword(_ text: String) -> String? {
text.lowercased()
.components(separatedBy: .alphanumerics.inverted)
.filter { $0.count > 4 && !Self.stopwords.contains($0) }
.first
}

// MARK: - Persistence

private func saveDreamReport(_ report: DreamReport) async {
let duration = report.finishedAt.map { $0.timeIntervalSince(report.startedAt) } ?? 0
try? await memory.observe(
category: "dream_report",
content: [
"duration_s": Int(duration),
"pruned": report.pruned,
"promoted": report.promoted,
"resolved": report.resolved,
"insights": report.insights,
"improvements": report.improvements
],
metadata: ["importance": 4]
)
}
}

```

```
// MARK: - Models

struct DreamReport {
let startedAt: Date
var finishedAt: Date?
var pruned: Int = 0
var promoted: Int = 0
var resolved: Int = 0
var phases: [String] = []
var insights: [String] = []
var improvements: [String] = []
}
```

◆ HermesIntegration.swift (141 lines)

```swift
import Foundation

// MARK: - HermesSession

/// Lightweight descriptor of one app session (open → background/close).
struct HermesSession {
let id: String
let startedAt: Date
var messageCount: Int = 0
var toolCallCount: Int = 0
}

// MARK: - HermesIntegration
//
// High-level API for the rest of AppClaw.
// Fully local — no HTTP, no server, no network permission needed.
//
// Additions over v1:
// - Session lifecycle: logSessionStart / logSessionEnd
// - messageCount / toolCallCount tracked per session
// - Exposes currentSession for context-aware UI

actor HermesIntegration {
static let shared = HermesIntegration()

private let memory = HermesMemory.shared
private let proactive = HermesProactiveEngine.shared
private let contextTracker = HermesContextTracker.shared

private(set) var lastError: Error?
private(set) var currentSession: HermesSession?

private init() {}

// MARK: - Session lifecycle

/// Call when a new conversation starts (app launch, new chat, scene activation).
func logSessionStart(conversationId: String) async {
let session = HermesSession(id: conversationId, startedAt: Date())
currentSession = session
await run {
try await self.memory.observe(
category: "session_start",
content: ["conversationId": conversationId],
metadata: ["importance": 2]
)
}
}

/// Call when the app backgrounds or a conversation ends.
func logSessionEnd() async {
guard let session = currentSession else { return }
currentSession = nil
await run {
try await self.memory.observe(
category: "session_end",
content: [
"conversationId": session.id,
"durationSeconds": Int(Date().timeIntervalSince(session.startedAt)),
"messageCount": session.messageCount,
"toolCallCount": session.toolCallCount
],
metadata: ["importance": 3]
)
}
}

// MARK: - Logging

func logUserMessage(_ text: String, in conversationId: String) async {
currentSession?.messageCount += 1
await contextTracker.ingest(text: text, role: .user)
await run {
try await self.memory.observe(
category: "user_message",
```

```
content: ["text": text, "conversationId": conversationId],
metadata: ["importance": 3]
)
}
}

func logAssistantResponse(_ text: String, toolUses: [String] = []) async {
await contextTracker.ingest(text: text, role: .assistant)
await run {
try await self.memory.observe(
category: "assistant_response",
content: ["text": text, "tools": toolUses]
)
}
}

func logToolExecution(_ toolName: String, success: Bool, duration: TimeInterval) async {
currentSession?.toolCallCount += 1
await run {
try await self.memory.observe(
category: "tool_exec",
content: [
"tool": toolName,
"success": success,
"duration_ms": Int(duration * 1000)
],
// Failures matter more: keep them longer, surface in DreamEngine
metadata: ["importance": success ? 1 : 3]
)
}
}

// MARK: - Queries

func pollSuggestions() async -> [HermesSuggestion] {
await proactive.currentSuggestions()
}

func currentTopic() async -> String? {
await contextTracker.currentTopic()
}

func searchMemory(query: String, limit: Int = 20) async -> [MemoryEntry] {
await memory.search(query: query, limit: limit)
}

func recentEvents(limit: Int = 50) async -> [MemoryEntry] {
await memory.recentEntries(limit: limit)
}

// MARK: - Private

@discardableResult
private func run(_ block: @escaping () async throws -> Void) async -> Bool {
do {
try await block()
return true
} catch {
self.lastError = error
#if DEBUG
print("[Hermes] \(error)")
#endif
return false
}
}
}
```

◆ HermesInterestEngine.swift (213 lines)

```swift
import Foundation
import UserNotifications

// MARK: - HermesInterestEngine
//
// Manages interest-based notifications and content.
// Uses only on-device scheduling (UNUserNotificationCenter) — no external
// tracking, fully App Store compliant.
//
// For real-time event data (sports scores, movie releases) the engine uses
// free public RSS/JSON APIs with no auth required:
// • Movies: RSS from iTunes Movie Trailers
// • Sports: ESPN public scores endpoint
// • News: RSS feeds per topic

actor HermesInterestEngine {
static let shared = HermesInterestEngine()

private let session = URLSession.shared

private init() {}

// MARK: - Notification permission

func requestPermission() async -> Bool {
do {
return try await UNUserNotificationCenter.current()
.requestAuthorization(options: [.alert, .sound, .badge])
} catch {
return false
}
}

// MARK: - Schedule interest notifications

/// Call after onboarding or whenever interests change.
func scheduleInterestNotifications(for persona: UserPersona) async {
guard await requestPermission() else { return }

// Remove all existing interest notifications before rescheduling
let ids = persona.interests.map { "interest_\($0.id)" }
UNUserNotificationCenter.current().removePendingNotificationRequests(withIdentifiers: ids)

for interest in persona.interests where interest.notificationsEnabled {
await scheduleNotification(for: interest, persona: persona)
}
}

private func scheduleNotification(for interest: Interest, persona: UserPersona) async {
let content = UNMutableNotificationContent()
content.title = "\(persona.assistantName) ■"
content.sound = .default

// Build body based on category — try to fetch real content, fall back to prompt
content.body = await notificationBody(for: interest, persona: persona)

// Schedule once daily at a sensible time per category
var comps = DateComponents()
comps.hour = notificationHour(for: interest.category)
comps.minute = 0

let trigger = UNCalendarNotificationTrigger(dateMatching: comps, repeats: true)
let request = UNNotificationRequest(
identifier: "interest_\(interest.id)",
content: content,
trigger: trigger
)
try? await UNUserNotificationCenter.current().add(request)
}

private func notificationHour(for category: Interest.Category) -> Int {
switch category {
case .sports: return 9 // morning scores
case .movies: return 18 // evening — after work
case .food: return 12 // lunch time
```

```
case .fitness: return 7 // morning workout nudge
case .finance: return 8 // pre-market
case .music: return 17 // after-school/work
default: return 10
}
}

// MARK: - Notification body generation

private func notificationBody(for interest: Interest, persona: UserPersona) async -> String {
let name = persona.userName.isEmpty ? "" : " \(persona.userName),"
let detail = interest.detail ?? interest.label

switch interest.category {
case .movies:
if let headline = await fetchMovieHeadline() {
return "■\(name) \(headline)"
}
return "■\(name) Any good movies on your radar? I found something you might like — ask me!"

case .sports:
if let score = await fetchSportsHeadline(team: interest.detail) {
return "■\(name) \(score)"
}
return "■\(name) Checking in on \(detail) — want the latest? Just ask me!"

case .music:
return "■\(name) New releases dropped this week. Want me to tell you about them?"

case .food:
if detail.lowercased().contains("starbucks") {
return "■■\(name) Ready for your \(detail)? I can help you reorder — just tap!"
}
return "■\(name) Thinking about what to eat? I have some ideas for you."

case .fitness:
return "■\(name) Time to move! Even 10 minutes counts. Want a quick routine?"

case .finance:
return "■\(name) Quick money check-in — want to log any spending today?"

case .tech:
return "■■\(name) Something interesting happened in tech today. Want the rundown?"

case .travel:
return "✈■\(name) Dreaming of your next trip? Ask me for inspiration!"

default:
return "Hey\(name) just thinking of you. Tap to chat!"
}
}

// MARK: - Light public API fetches (no auth, no tracking)

private func fetchMovieHeadline() async -> String? {
guard let url = URL(string: "https://trailers.apple.com/trailers/home/rss/newtrailers.rss") else { return nil }
guard let (data, _) = try? await session.data(from: url) else { return nil }
// Parse first <title> after the channel title from RSS
let xml = String(data: data, encoding: .utf8) ?? ""
let titles = xml.components(separatedBy: "<title>")
.dropFirst(2) // skip channel title and first item
.prefix(1)
.compactMap { $0.components(separatedBy: "</title>").first }
.map { $0.trimmingCharacters(in: .whitespacesAndNewlines) }
return titles.first.map { "New trailer: \($0)" }
}

private func fetchSportsHeadline(team: String?) async -> String? {
// ESPN public scoreboard — no API key needed
let today = ISO8601DateFormatter().string(from: Date()).prefix(10)
guard let url = URL(string: "https://site.api.espn.com/apis/site/v2/sports/basketball/nba/scoreboard?dates=\(…
guard let (data, _) = try? await session.data(from: url),
let json = try? JSONSerialization.jsonObject(with: data) as? [String: Any],
let events = json["events"] as? [[String: Any]] else { return nil }

for event in events {
let name = (event["name"] as? String) ?? ""
```

```
if let team, name.lowercased().contains(team.lowercased()) {
let status = ((event["status"] as? [String: Any])?["type"] as? [String: Any])?["description"] as? Str…
let competitors = (event["competitions"] as? [[String: Any]])?.first?["competitors"] as? [[String: An…
let scores = competitors.compactMap { c -> String? in
guard let t = (c["team"] as? [String: Any])?["abbreviation"] as? String,
let s = c["score"] as? String else { return nil }
return "\(t) \(s)"
}.joined(separator: " – ")
return "\(name): \(scores) (\(status))"
}
}
return nil
}

// MARK: - Send immediate interest notification (for chat-triggered updates)

func sendImmediateNotification(title: String, body: String, identifier: String = UUID().uuidString) {
let content = UNMutableNotificationContent()
content.title = title
content.body = body
content.sound = .default

let trigger = UNTimeIntervalNotificationTrigger(timeInterval: 1, repeats: false)
let request = UNNotificationRequest(identifier: identifier, content: content, trigger: trigger)
UNUserNotificationCenter.current().add(request)
}

// MARK: - Detect new interests from conversation

/// Parse a user message for newly mentioned interests and return any found.
func detectInterests(in text: String) -> [Interest] {
let lower = text.lowercased()
var found: [Interest] = []

let checks: [(keyword: String, interest: Interest)] = [
("movie", Interest(id: "movies", category: .movies, label: "Movies", emoji: "■")),
("film", Interest(id: "movies", category: .movies, label: "Movies", emoji: "■")),
("netflix", Interest(id: "movies", category: .movies, label: "Movies", emoji: "■")),
("nba", Interest(id: "sports_nba", category: .sports, label: "NBA", emoji: "■")),
("nfl", Interest(id: "sports_nfl", category: .sports, label: "NFL", emoji: "■")),
("mlb", Interest(id: "sports_mlb", category: .sports, label: "MLB", emoji: "■■")),
("soccer",Interest(id: "sports_soccer", category: .sports, label: "Soccer", emoji: "■■")),
("music", Interest(id: "music", category: .music, label: "Music", emoji: "■")),
("spotify",Interest(id: "music", category: .music, label: "Music", emoji: "■")),
("gym", Interest(id: "fitness", category: .fitness, label: "Fitness", emoji: "■")),
("workout",Interest(id: "fitness", category: .fitness, label: "Fitness", emoji: "■")),
("starbucks", Interest(id: "food_starbucks", category: .food, label: "Starbucks", emoji: "■■")),
("coffee", Interest(id: "food_coffee", category: .food, label: "Coffee", emoji: "■■")),
("travel", Interest(id: "travel", category: .travel, label: "Travel", emoji: "■")),
("gaming", Interest(id: "gaming", category: .gaming, label: "Gaming", emoji: "■")),
("crypto", Interest(id: "finance_crypto", category: .finance, label: "Crypto", emoji: "■")),
("stocks", Interest(id: "finance_stocks", category: .finance, label: "Stocks", emoji: "■")),
]

for check in checks where lower.contains(check.keyword) {
found.append(check.interest)
}

return found
}
}
```

◆ HermesKairos.swift (290 lines)

```swift
import Foundation

// MARK: - Kairos ("the right time")
//
// Kairos is the always-on background observer. It watches the user's working
// environment, writes structured observation logs, and waits for a 15-second
// window of inactivity before proposing or taking autonomous action — so it
// never interrupts mid-thought.
//
// Three design contracts:
// 1. 15-Second Budget — only act after 15 s of detected user inactivity.
// 2. Observation Log — every environmental scan is recorded before any
// action is proposed, creating an audit trail.
// 3. Strict Write Discipline — observations are only committed to persistent
// memory AFTER the related action succeeds. Failed
// or speculative data is never stored.

// MARK: - Models

struct KairosObservation {
let id: UUID
let timestamp: Date
let trigger: ObservationTrigger
let context: [String: Any] // freeform snapshot of the environment
let sessionTopic: String?
let intent: HermesContextTracker.Intent?

enum ObservationTrigger: String {
case idleTimeout // 15 s of inactivity elapsed
case sessionStart // app foregrounded
case sessionEnd // app backgrounded
case errorDetected // tool failure logged
case patternRecognised // DreamEngine surfaced an insight
}
}

struct KairosAction {
let id: UUID
let observation: KairosObservation
let type: ActionType
let description: String
let confidence: Double // 0.0–1.0; only execute if >= threshold
var outcome: ActionOutcome?

enum ActionType: String {
case suggestContext // surface a suggestion to the UI
case consolidateMemory // trigger early mini-dream
case flagContradiction // mark conflicting memories for review
case promoteEntry // bump importance of a relevant entry
case logInsight // write a new dream_insight entry
}

enum ActionOutcome {
case success
case failure(Error)
case skipped(reason: String)
}
}

// MARK: - HermesKairos

actor HermesKairos {
static let shared = HermesKairos()

// Dependencies
private let memory = HermesMemory.shared
private let context = HermesContextTracker.shared
private let proactive = HermesProactiveEngine.shared

// State
private var isRunning = false
private var lastActivity: Date = Date()
private var observationLog: [KairosObservation] = []
private var pendingWrites: [(category: String, content: Any, metadata: [String: Any])] = []
// Bug fix #3: store the loop Task so pause() can cancel it and stop the chain.
```

```
private var loopTask: Task<Void, Never>?

// Config
private let idleBudgetSeconds: TimeInterval = 15
private let minConfidenceToAct: Double = 0.65
private let maxObservationLogSize = 200

private init() {}

// MARK: - Lifecycle

/// Start the Kairos background loop. Call from AppDelegate / scene activation.
func start() {
guard !isRunning else { return }
isRunning = true
scheduleNextCheck()
}

/// Pause Kairos (e.g. when app backgrounds — DreamEngine takes over instead).
func pause() {
isRunning = false
// Bug fix #3: cancel the stored task so the recursive chain terminates.
loopTask?.cancel()
loopTask = nil
}

/// Signal that the user is active — resets the 15-second idle budget.
func userDidAct() {
lastActivity = Date()
}

// MARK: - Idle check loop

private func scheduleNextCheck() {
// Bug fix #3: store the task so pause() can cancel it.
loopTask = Task { [weak self] in
// Poll every 5 s; fire the full observation when idle budget is met
try? await Task.sleep(nanoseconds: 5_000_000_000)
guard !Task.isCancelled, let self, await self.isRunning else { return }
await self.tick()
await self.scheduleNextCheck()
}
}

private func tick() async {
let idle = Date().timeIntervalSince(lastActivity)
guard idle >= idleBudgetSeconds else { return }

let obs = await buildObservation(trigger: .idleTimeout)
observationLog.append(obs)
trimObservationLog()

let action = await proposeAction(for: obs)
await executeIfWarranted(action)
}

// MARK: - Observation

private func buildObservation(trigger: KairosObservation.ObservationTrigger) async -> KairosObservation {
let topic = await context.currentTopic()
let intent = await context.currentIntent()
let recent = await memory.recentEntries(limit: 5)

let ctx: [String: Any] = [
"recentCategories": recent.map(\.category),
"idleSeconds": Date().timeIntervalSince(lastActivity),
"observationCount": observationLog.count
]

return KairosObservation(
id: UUID(),
timestamp: Date(),
trigger: trigger,
context: ctx,
sessionTopic: topic,
intent: intent
)
```

```
}

// MARK: - Action proposal

private func proposeAction(for obs: KairosObservation) async -> KairosAction {
// Check for tool failures → flag or consolidate
let recentErrors = await memory.entries(for: "tool_exec")
.prefix(20)
.filter {
guard let d = $0.content.value as? [String: Any],
let ok = d["success"] as? Bool else { return false }
return !ok
}

if recentErrors.count >= 3 {
return KairosAction(
id: UUID(), observation: obs,
type: .consolidateMemory,
description: "Recurring failures detected — trigger early memory consolidation.",
confidence: 0.80
)
}

// Check if a dream insight exists that hasn't been surfaced yet
let unseenInsights = await memory.entries(for: "dream_insight")
.filter { $0.timestamp > Date().addingTimeInterval(-3600) }
if !unseenInsights.isEmpty {
return KairosAction(
id: UUID(), observation: obs,
type: .suggestContext,
description: "Unseen dream insight available — refresh proactive suggestions.",
confidence: 0.72
)
}

// Default: nothing warranted right now
return KairosAction(
id: UUID(), observation: obs,
type: .suggestContext,
description: "Idle scan complete, no urgent action.",
confidence: 0.10
)
}

// MARK: - Execution with strict write discipline

private func executeIfWarranted(_ action: KairosAction) async {
guard action.confidence >= minConfidenceToAct else {
recordOutcome(action, .skipped(reason: "Confidence \(action.confidence) below threshold \(minConfidenceTo…
return
}

var result = action
do {
try await perform(action)
// ■ STRICT WRITE DISCIPLINE: only flush pending writes after success
try await flushPendingWrites()
result.outcome = .success
} catch {
// ■ Do NOT write speculative data — discard pending writes
pendingWrites.removeAll()
result.outcome = .failure(error)
}
recordOutcome(result, result.outcome ?? .skipped(reason: "unknown"))
}

private func perform(_ action: KairosAction) async throws {
switch action.type {
case .consolidateMemory:
try await memory.consolidate(importanceThreshold: 2, keepWindow: 7 * 86400)

case .suggestContext:
await proactive.refresh()

case .promoteEntry:
// Promote the most recently accessed relevant entry
let recent = await memory.recentEntries(limit: 1)
```

```
if let entry = recent.first {
try await memory.promoteToLongTerm([entry.id])
}

case .logInsight:
let topic = action.observation.sessionTopic ?? "general"
// Stage the write — only committed if this action succeeds (flushPendingWrites)
pendingWrites.append((
category: "kairos_insight",
content: ["topic": topic, "trigger": action.observation.trigger.rawValue],
metadata: ["importance": 3]
))

case .flagContradiction:
// Surfaced by DreamEngine phase 5; log for review
pendingWrites.append((
category: "kairos_flag",
content: ["description": action.description],
metadata: ["importance": 4]
))
}
}

/// Flush staged writes to persistent memory — only called on success.
private func flushPendingWrites() async throws {
for write in pendingWrites {
try await memory.observe(category: write.category,
content: write.content,
metadata: write.metadata)
}
pendingWrites.removeAll()
}

private func recordOutcome(_ action: KairosAction, _ outcome: KairosAction.ActionOutcome) {
// Lightweight in-memory only — no disk write for outcome records
// (they'd create noise; the observation log is the audit trail)
#if DEBUG
let status: String
switch outcome {
case .success: status = "✓"
case .failure(let e): status = "✗ \(e)"
case .skipped(let r): status = "– \(r)"
}
print("[Kairos] \(action.type.rawValue) \(status)")
#endif
}

// MARK: - Observation log management

private func trimObservationLog() {
if observationLog.count > maxObservationLogSize {
observationLog.removeFirst(observationLog.count - maxObservationLogSize)
}
}

// MARK: - Public accessors

func recentObservations(limit: Int = 20) -> [KairosObservation] {
Array(observationLog.suffix(limit).reversed())
}
}
```

◆ HermesLLMClient.swift (500 lines)

```swift
import Foundation

// MARK: - LLM Provider

/// Which backend is being used for this session.
enum LLMProvider {
case appleFoundationModels // on-device, iOS 26+, no privacy policy needed
case claudeAPI // remote Anthropic API, requires user consent
case none // no provider available / consent not given
}

// MARK: - LLM Request / Response

struct LLMRequest {
let systemPrompt: String
let messages: [LLMMessage]
let tools: [ToolDefinition]
let maxTokens: Int
let role: AgentRole
}

struct LLMMessage {
enum Role { case user, assistant, system }
let role: Role
let content: String
}

struct LLMResponse {
let content: String
let promptTokens: Int
let completionTokens: Int
let provider: LLMProvider
let toolCallsRequested: [String] // tool IDs the model wants to invoke
}

// MARK: - Streaming handler

typealias StreamHandler = @Sendable (String) -> Void

// MARK: - HermesLLMClient

/// Single entry point for all LLM calls.
/// Automatically routes to Apple Foundation Models or Claude API based on
/// device capability and user consent. Either way, Hermes context is
/// injected into every request before it leaves this class.
actor HermesLLMClient {
static let shared = HermesLLMClient()

private let memory = HermesMemory.shared
private let context = HermesContextTracker.shared
private let session = HermesSessionState.shared
private let privacy = HermesPrivacyGate.shared

private var _provider: LLMProvider = .none

private init() {}

// MARK: - Provider selection

/// Call once at app start (after privacy gate resolves).
func configure() async {
if await privacy.consentGiven {
_provider = Self.bestAvailableProvider()
} else {
_provider = .none
}
}

var provider: LLMProvider { _provider }

private static func bestAvailableProvider() -> LLMProvider {
// iOS 26+ with Apple Intelligence available → prefer on-device
if #available(iOS 26.0, *) {
if AppleFoundationModelsBridge.isAvailable {
return .appleFoundationModels
```

```
}
}
// Fall back to Claude API (requires API key configured)
if ClaudeAPIBridge.isConfigured {
return .claudeAPI
}
return .none
}

// MARK: - Main call (with full Hermes context injection)

/// Complete an LLM turn. Hermes memory and session context are
/// automatically injected into the system prompt before sending.
func complete(request: LLMRequest,
stream: StreamHandler? = nil) async throws -> LLMResponse {
guard _provider != .none else {
throw LLMError.noProviderConfigured
}

// Pre-turn budget check
try await session.checkBudget(estimatedCost: request.maxTokens)

// Build Hermes-enriched system prompt
let enrichedSystem = await buildSystemPrompt(base: request.systemPrompt,
role: request.role)

let enrichedRequest = LLMRequest(
systemPrompt: enrichedSystem,
messages: request.messages,
tools: request.tools,
maxTokens: request.maxTokens,
role: request.role
)

// Route to provider
let response: LLMResponse
switch _provider {
case .appleFoundationModels:
response = try await AppleFoundationModelsBridge.complete(enrichedRequest, stream: stream)
case .claudeAPI:
response = try await ClaudeAPIBridge.complete(enrichedRequest, stream: stream)
case .none:
throw LLMError.noProviderConfigured
}

// Record token usage (strict write discipline: only on success)
try await session.recordTokenUsage(prompt: response.promptTokens,
completion: response.completionTokens)
return response
}

// MARK: - Hermes context injection

/// Builds the system prompt enriched with:
/// • Role-specific instructions
/// • Current session topic and intent
/// • Relevant memories from the index
/// • Recent dream insights
/// • Self-improvement notes
private func buildSystemPrompt(base: String, role: AgentRole) async -> String {
var sections: [String] = []

// 1. Base instructions + role constraints
sections.append(base)
sections.append(roleInstructions(role))

// 2. Current session context (lightweight — from ContextTracker, no disk read)
if let topic = await context.currentTopic() {
sections.append("## Current topic\n\(topic)")
}
if let intent = await context.currentIntent() {
sections.append("## Detected intent\n\(intent.rawValue)")
}
let summary = await context.sessionSummary()
if summary != "No activity yet." {
sections.append("## Recent conversation\n\(summary)")
}
```

```
153
154 // 3. Relevant long-term memories (via MemoryIndex — no full deserialisation)
155 if let topic = await context.currentTopic() {
156 let keywords = topic.components(separatedBy: ", ")
157 let relevant = await memory.indexSearch(keywords: keywords, limit: 5)
158 if !relevant.isEmpty {
159 let bullets = relevant.compactMap { entry -> String? in
160 guard let text = (entry.content.value as? [String: Any])
161 .flatMap({ $0.values.compactMap { $0 as? String }.first })
162 ?? (entry.content.value as? String)
163 else { return nil }
164 return "• [\(entry.category)] \(text.prefix(200))"
165 }.joined(separator: "\n")
166 sections.append("## Relevant memory\n\(bullets)")
167 }
168 }
169
170 // 4. Recent dream insights (from last 24 h)
171 let insights = await memory.entries(for: "dream_insight")
172 .prefix(2)
173 .compactMap { ($0.content.value as? [String: Any])?["insight"] as? String }
174 if !insights.isEmpty {
175 sections.append("## Recent insights\n" + insights.map { "• \($0)" }.joined(separator: "\n"))
176 }
177
178 // 5. Self-improvement notes
179 let improvements = await memory.entries(for: "self_improvement")
180 .prefix(1)
181 .compactMap { ($0.content.value as? [String: Any])?["note"] as? String }
182 if !improvements.isEmpty {
183 sections.append("## Known issues to avoid\n" + improvements.map { "• \($0)" }.joined(separator: "\n"))
184 }
185
186 return sections.joined(separator: "\n\n")
187 }
188
189 private func roleInstructions(_ role: AgentRole) -> String {
190 switch role {
191 case .explore:
192 return "## Role: Explorer\nRead and investigate only. Do NOT modify any state. Summarise findings clearly…
193 case .plan:
194 return "## Role: Planner\nSynthesise the explorer's findings into a clear, numbered action plan. No side …
195 case .execute:
196 return "## Role: Executor\nCarry out the plan step by step. Confirm each step before moving to the next. …
197 case .verify:
198 return "## Role: Verifier\nCritically review the executor's output. Flag any discrepancies, incomplete st…
199 }
200 }
201 }
202
203 // MARK: - Error
204
205 enum LLMError: Error, LocalizedError {
206 case noProviderConfigured
207 case consentNotGiven
208 case apiKeyMissing
209 case rateLimited
210 case contextTooLong
211
212 var errorDescription: String? {
213 switch self {
214 case .noProviderConfigured: return "No AI provider configured. Please check Settings."
215 case .consentNotGiven: return "AI features require your consent. See Settings → Privacy."
216 case .apiKeyMissing: return "Claude API key not set. Add it in Settings."
217 case .rateLimited: return "Too many requests. Please wait a moment."
218 case .contextTooLong: return "Conversation too long. Starting a new session."
219 }
220 }
221 }
222
223 // MARK: - Apple Foundation Models bridge (iOS 26+)
224 //
225 // Requires:
226 // 1. Xcode with iOS 26 SDK (Xcode 26 beta or later)
227 // 2. Add FoundationModels.framework to target → Frameworks, Libraries,
228 // and Embedded Content (it ships with the iOS 26 SDK, no entitlement needed)
229 // 3. Deployment target can stay at iOS 17+ — all API calls are guarded
```

```
// with #available(iOS 26.0, *) at runtime

#if canImport(FoundationModels)
import FoundationModels
#endif

enum AppleFoundationModelsBridge {

// MARK: - Availability

/// True when Apple Intelligence is supported AND enabled on this device.
static var isAvailable: Bool {
#if canImport(FoundationModels)
if #available(iOS 26.0, *) {
switch SystemLanguageModel.default.availability {
case .available: return true
case .unavailable: return false // .deviceNotEligible / .appleIntelligenceNotEnabled / .modelNotReady
}
}
#endif
return false
}

// MARK: - Completion

static func complete(_ request: LLMRequest,
stream: StreamHandler?) async throws -> LLMResponse {
#if canImport(FoundationModels)
if #available(iOS 26.0, *) {
return try await _complete26(request, stream: stream)
}
#endif
throw LLMError.noProviderConfigured
}

// MARK: - iOS 26 implementation (compiled only when SDK available)

#if canImport(FoundationModels)
@available(iOS 26.0, *)
private static func _complete26(_ request: LLMRequest,
stream: StreamHandler?) async throws -> LLMResponse {
guard case .available = SystemLanguageModel.default.availability else {
throw LLMError.noProviderConfigured
}

// One session per agent run — holds conversation state internally.
let session = LanguageModelSession(instructions: request.systemPrompt)
let composed = composePrompt(from: request.messages)

var fullText = ""
let estimatedPromptTokens = composed.count / 4

if let handler = stream {
for try await partial in session.streamResponse(to: composed) {
handler(partial.content)
fullText += partial.content
}
} else {
let response = try await session.respond(to: composed)
fullText = response.content
}

return LLMResponse(
content: fullText,
promptTokens: estimatedPromptTokens,
completionTokens: fullText.count / 4,
provider: .appleFoundationModels,
toolCallsRequested: []
)
}

private static func composePrompt(from messages: [LLMMessage]) -> String {
messages
.filter { $0.role != .system }
.map { msg -> String in
switch msg.role {
case .user: return "User: \(msg.content)"
```

```
case .assistant: return "Assistant: \(msg.content)"
case .system: return ""
}
}
.filter { !$0.isEmpty }
.joined(separator: "\n\n")
}
#endif
}

// MARK: - Claude API bridge

/// Calls the Anthropic Messages API.
/// Requires ANTHROPIC_API_KEY in the keychain (never in source or Info.plist).
enum ClaudeAPIBridge {

private static let endpoint = URL(string: "https://api.anthropic.com/v1/messages")!
private static let apiVersion = "2023-06-01"

static var isConfigured: Bool {
apiKey != nil
}

private static var apiKey: String? {
// Read from Keychain — never hardcode
KeychainHelper.read(service: "com.openclaw.appclaw", key: "anthropic_api_key")
}

static func complete(_ request: LLMRequest,
stream: StreamHandler?) async throws -> LLMResponse {
guard let key = apiKey else { throw LLMError.apiKeyMissing }

// Select model per agent role
let model = modelForRole(request.role)

// Build Anthropic messages array from transcript
let messages: [[String: Any]] = request.messages.compactMap { msg in
guard msg.role != .system else { return nil } // system goes in top-level key
return ["role": msg.role == .user ? "user" : "assistant",
"content": msg.content]
}

// Build tools array from ToolDefinitions
let tools: [[String: Any]] = request.tools.map { tool in
[
"name": tool.id.replacingOccurrences(of: ".", with: "_"),
"description": tool.description,
"input_schema": ["type": "object", "properties": tool.inputSchema]
]
}

var body: [String: Any] = [
"model": model,
"max_tokens": request.maxTokens,
"system": request.systemPrompt,
"messages": messages,
]
if !tools.isEmpty { body["tools"] = tools }
if stream != nil { body["stream"] = true }

var urlRequest = URLRequest(url: endpoint)
urlRequest.httpMethod = "POST"
urlRequest.setValue("application/json", forHTTPHeaderField: "Content-Type")
urlRequest.setValue(key, forHTTPHeaderField: "x-api-key")
urlRequest.setValue(apiVersion, forHTTPHeaderField: "anthropic-version")
urlRequest.httpBody = try JSONSerialization.data(withJSONObject: body)

if let handler = stream {
return try await streamResponse(urlRequest, handler: handler)
} else {
return try await blockingResponse(urlRequest)
}
}

// MARK: Streaming

private static func streamResponse(_ request: URLRequest,
```

```
handler: @escaping StreamHandler) async throws -> LLMResponse {
let (bytes, response) = try await URLSession.shared.bytes(for: request)
try validateHTTP(response)

var fullText = ""
var inputTokens = 0
var outputTokens = 0

for try await line in bytes.lines {
guard line.hasPrefix("data: ") else { continue }
let jsonStr = String(line.dropFirst(6))
guard jsonStr != "[DONE]",
let data = jsonStr.data(using: .utf8),
let event = try? JSONSerialization.jsonObject(with: data) as? [String: Any]
else { continue }

// Text delta
if let delta = (event["delta"] as? [String: Any])?["text"] as? String {
handler(delta)
fullText += delta
}
// Usage (arrives in message_delta event)
if let usage = event["usage"] as? [String: Any] {
inputTokens = usage["input_tokens"] as? Int ?? inputTokens
outputTokens = usage["output_tokens"] as? Int ?? outputTokens
}
}

return LLMResponse(content: fullText, promptTokens: inputTokens,
completionTokens: outputTokens,
provider: .claudeAPI, toolCallsRequested: [])
}

// MARK: Blocking

private static func blockingResponse(_ request: URLRequest) async throws -> LLMResponse {
let (data, response) = try await URLSession.shared.data(for: request)
try validateHTTP(response)

guard let json = try JSONSerialization.jsonObject(with: data) as? [String: Any],
let content = (json["content"] as? [[String: Any]])?.first,
let text = content["text"] as? String
else { throw LLMError.noProviderConfigured }

let usage = json["usage"] as? [String: Any]
let input = usage?["input_tokens"] as? Int ?? 0
let output = usage?["output_tokens"] as? Int ?? 0

// Extract any tool_use blocks
let toolCalls = (json["content"] as? [[String: Any]] ?? [])
.filter { $0["type"] as? String == "tool_use" }
.compactMap { $0["name"] as? String }

return LLMResponse(content: text, promptTokens: input,
completionTokens: output,
provider: .claudeAPI, toolCallsRequested: toolCalls)
}

private static func validateHTTP(_ response: URLResponse) throws {
guard let http = response as? HTTPURLResponse else { return }
switch http.statusCode {
case 200...299: return
case 429: throw LLMError.rateLimited
default: throw LLMError.noProviderConfigured
}
}

private static func modelForRole(_ role: AgentRole) -> String {
switch role {
case .explore: return "claude-haiku-4-5-20251001" // fast + cheap for reads
case .plan: return "claude-sonnet-4-6" // balanced
case .execute: return "claude-sonnet-4-6" // reliable for writes
case .verify: return "claude-opus-4-6" // highest scrutiny
}
}
}

```

```swift
// MARK: - Keychain helper

enum KeychainHelper {
static func read(service: String, key: String) -> String? {
let query: [CFString: Any] = [
kSecClass: kSecClassGenericPassword,
kSecAttrService: service,
kSecAttrAccount: key,
kSecReturnData: true,
kSecMatchLimit: kSecMatchLimitOne,
]
var result: AnyObject?
let status = SecItemCopyMatching(query as CFDictionary, &result)
guard status == errSecSuccess,
let data = result as? Data,
let string = String(data: data, encoding: .utf8)
else { return nil }
return string
}

static func write(service: String, key: String, value: String) {
let data = value.data(using: .utf8)!
// Delete existing first
let deleteQuery: [CFString: Any] = [
kSecClass: kSecClassGenericPassword,
kSecAttrService: service,
kSecAttrAccount: key,
]
SecItemDelete(deleteQuery as CFDictionary)
// Add new
let addQuery: [CFString: Any] = [
kSecClass: kSecClassGenericPassword,
kSecAttrService: service,
kSecAttrAccount: key,
kSecValueData: data,
kSecAttrAccessible: kSecAttrAccessibleWhenUnlockedThisDeviceOnly,
]
SecItemAdd(addQuery as CFDictionary, nil)
}
}
```

◆ HermesMemory.swift (413 lines)

```swift
import Foundation

// MARK: - Memory tier

/// Short-term holds raw, recent observations.
/// Long-term holds entries that have been promoted by the DreamEngine.
enum MemoryTier: String, Codable {
case shortTerm
case longTerm
}

// MARK: - MemoryEntry

struct MemoryEntry: Codable, Identifiable {
let id: UUID
let timestamp: Date
let category: String
let content: AnyCodable
let metadata: [String: AnyCodable]
var importance: Int // 1–5; decays over time, boosted by DreamEngine
var tier: MemoryTier
var accessCount: Int // incremented on retrieval; informs eviction

init(
id: UUID = UUID(),
timestamp: Date = Date(),
category: String,
content: Any,
metadata: [String: Any] = [:],
importance: Int = 1,
tier: MemoryTier = .shortTerm
) {
self.id = id
self.timestamp = timestamp
self.category = category
self.content = AnyCodable(content)
self.metadata = metadata.mapValues { AnyCodable($0) }
self.importance = max(1, min(5, importance))
self.tier = tier
self.accessCount = 0
}
}

// MARK: - AnyCodable

/// Type-erased Codable wrapper for arbitrary JSON-compatible values.
struct AnyCodable: Codable {
let value: Any

init(_ value: Any) { self.value = value }

init(from decoder: Decoder) throws {
let c = try decoder.singleValueContainer()
// Bool must come before Int: in JSON, true/false are distinct from 1/0
if let v = try? c.decode(Bool.self) { value = v; return }
if let v = try? c.decode(Int.self) { value = v; return }
if let v = try? c.decode(Double.self) { value = v; return }
if let v = try? c.decode(String.self) { value = v; return }
if let v = try? c.decode([AnyCodable].self) { value = v.map(\.value); return }
if let v = try? c.decode([String: AnyCodable].self) { value = v.mapValues(\.value); return }
value = NSNull()
}

func encode(to encoder: Encoder) throws {
var c = encoder.singleValueContainer()
switch value {
case let v as Bool: try c.encode(v)
case let v as Int: try c.encode(v)
case let v as Double: try c.encode(v)
case let v as String: try c.encode(v)
case let v as [Any]: try c.encode(v.map { AnyCodable($0) })
case let v as [String: Any]: try c.encode(v.mapValues { AnyCodable($0) })
default: try c.encodeNil()
}
}
```

```swift
}

// MARK: - MemoryIndex (Layered Memory Architecture)
//
// A lightweight keyword → entry-ID map persisted separately from the full
// entry store. The HermesAgentHarness queries this index first; it only
// deserialises full entries for the small result set, preventing context
// entropy from reloading entire conversation histories.

struct MemoryIndex: Codable {
/// keyword → set of entry IDs that contain it
var keywordMap: [String: [UUID]] = [:]
var updatedAt: Date = Date()

/// Add an entry's keywords to the index.
mutating func index(entry: MemoryEntry) {
let words = tokenise("\(entry.content.value) \(entry.category)")
for word in words {
var ids = keywordMap[word, default: []]
if !ids.contains(entry.id) { ids.append(entry.id) }
keywordMap[word] = ids
}
}

/// Remove an entry's ID from all keyword buckets.
mutating func remove(id: UUID) {
for key in keywordMap.keys {
keywordMap[key]?.removeAll { $0 == id }
}
}

/// Look up entry IDs matching ALL of the given keywords (AND logic).
func lookup(keywords: [String]) -> Set<UUID> {
guard !keywords.isEmpty else { return [] }
let sets = keywords.map { kw -> Set<UUID> in
Set(keywordMap[kw.lowercased()] ?? [])
}
return sets.dropFirst().reduce(sets[0]) { $0.intersection($1) }
}

/// Look up entry IDs matching ANY keyword (OR logic).
func lookupAny(keywords: [String]) -> Set<UUID> {
Set(keywords.flatMap { keywordMap[$0.lowercased()] ?? [] })
}

private func tokenise(_ text: String) -> [String] {
text.lowercased()
.components(separatedBy: .alphanumerics.inverted)
.filter { $0.count > 3 }
}
}

// MARK: - HermesMemory

/// Fully on-device, sandboxed memory store.
///
/// v3 additions:
/// - MemoryIndex: lightweight keyword map persisted separately for O(1) search
/// without deserialising full entry payloads (layered memory architecture)
actor HermesMemory {
static let shared = HermesMemory()

// MARK: - Constants

private let shortTermCap = 2_000
private let decayIntervalDays: Double = 3
private let persistDebounceSeconds: Double = 2.0

// MARK: - Storage

private var entries: [MemoryEntry] = []
/// Category → entry IDs (maintained in sync with `entries`)
private var categoryIndex: [String: [UUID]] = [:]
/// Lightweight keyword index (separate file, fast to load)
private var memoryIndex: MemoryIndex = MemoryIndex()
private var loaded = false

```

```swift
/// Pending debounced persist task
private var pendingPersistTask: Task<Void, Never>?

// MARK: - Paths

private let memoryDir: URL = {
FileManager.default.urls(for: .documentDirectory, in: .userDomainMask)[0]
.appendingPathComponent("hermes", isDirectory: true)
}()

private var shortTermFile: URL { memoryDir.appendingPathComponent("short_term.json") }
private var longTermFile: URL { memoryDir.appendingPathComponent("long_term.json") }
private var indexFile: URL { memoryDir.appendingPathComponent("memory_index.json") }

// MARK: - Encoder / Decoder

private let encoder: JSONEncoder = {
let e = JSONEncoder()
e.dateEncodingStrategy = .iso8601
e.outputFormatting = .prettyPrinted
return e
}()

private let decoder: JSONDecoder = {
let d = JSONDecoder()
d.dateDecodingStrategy = .iso8601
return d
}()

private init() {}

// MARK: - Write

/// Record a new observation. Returns the created entry's ID.
@discardableResult
func observe(category: String, content: Any, metadata: [String: Any] = [:]) async throws -> UUID {
try ensureDir()
await loadIfNeeded()

let importance = (metadata["importance"] as? Int) ?? 1
let entry = MemoryEntry(category: category, content: content,
metadata: metadata, importance: importance)
append(entry)
applyDecay()
evictIfNeeded()
schedulePersist()
return entry.id
}

/// Batch-update multiple entries in one persist round-trip.
func updateBatch(_ updated: [MemoryEntry]) async throws {
await loadIfNeeded()
for entry in updated {
if let idx = entries.firstIndex(where: { $0.id == entry.id }) {
entries[idx] = entry
}
}
try await persistNow()
}

/// Update a single entry.
func update(_ entry: MemoryEntry) async throws {
try await updateBatch([entry])
}

// MARK: - Read

/// All entries, newest first.
func allEntries() async -> [MemoryEntry] {
await loadIfNeeded()
return entries.reversed()
}

/// Most recent N entries, newest first.
func recentEntries(limit: Int = 50) async -> [MemoryEntry] {
await loadIfNeeded()
return Array(entries.suffix(limit).reversed())
```

```
}

/// Entries in a given category, newest first — O(index) lookup.
func entries(for category: String) async -> [MemoryEntry] {
await loadIfNeeded()
let ids = Set(categoryIndex[category] ?? [])
return entries
.filter { ids.contains($0.id) }
.reversed()
}

/// Entries matching multiple categories.
func entries(forAny categories: [String]) async -> [MemoryEntry] {
await loadIfNeeded()
let ids = categories.flatMap { categoryIndex[$0] ?? [] }
let idSet = Set(ids)
return entries.filter { idSet.contains($0.id) }.reversed()
}

/// Keyword search across serialised content and category name.
func search(query: String, limit: Int = 20) async -> [MemoryEntry] {
await loadIfNeeded()
let q = query.lowercased()
let matches = entries
.filter { "\($0.content.value)".lowercased().contains(q)
|| $0.category.lowercased().contains(q) }
// Bump access count for found entries
let ids = Set(matches.map(\.id))
for i in entries.indices where ids.contains(entries[i].id) {
entries[i].accessCount += 1
}
return Array(matches.suffix(limit).reversed())
}

// MARK: - Maintenance

/// Prune low-importance, old short-term entries (called by DreamEngine).
func consolidate(importanceThreshold: Int = 2, keepWindow: TimeInterval = 7 * 86400) async throws {
await loadIfNeeded()
let cutoff = Date().addingTimeInterval(-keepWindow)
let before = entries.count
entries = entries.filter {
$0.tier == .longTerm
|| $0.importance >= importanceThreshold
|| $0.timestamp > cutoff
}
rebuildCategoryIndex()
if entries.count != before { try await persistNow() }
}

/// Promote entries to long-term tier.
func promoteToLongTerm(_ ids: [UUID]) async throws {
await loadIfNeeded()
var changed = false
for i in entries.indices where ids.contains(entries[i].id) {
if entries[i].tier != .longTerm {
entries[i].tier = .longTerm
changed = true
}
}
if changed { try await persistNow() }
}

// MARK: - Private helpers

private func append(_ entry: MemoryEntry) {
entries.append(entry)
categoryIndex[entry.category, default: []].append(entry.id)
memoryIndex.index(entry: entry)
}

private func rebuildCategoryIndex() {
categoryIndex = [:]
memoryIndex = MemoryIndex()
for entry in entries {
categoryIndex[entry.category, default: []].append(entry.id)
memoryIndex.index(entry: entry)
```

```
}
}

// MARK: - Index-based search (layered architecture: no full deserialisation)

/// Fast keyword search via MemoryIndex — resolves only matching entry IDs.
func indexSearch(keywords: [String], limit: Int = 20) async -> [MemoryEntry] {
await loadIfNeeded()
let ids = memoryIndex.lookupAny(keywords: keywords.map { $0.lowercased() })
let idMap = Dictionary(uniqueKeysWithValues: entries.map { ($0.id, $0) })
return Array(ids
.compactMap { idMap[$0] }
.sorted { $0.timestamp > $1.timestamp }
.prefix(limit))
}

/// Importance decay: short-term entries lose 1 point per `decayIntervalDays` days of age.
private func applyDecay() {
let now = Date()
for i in entries.indices where entries[i].tier == .shortTerm {
let ageDays = now.timeIntervalSince(entries[i].timestamp) / 86400
let decaySteps = Int(ageDays / decayIntervalDays)
let decayed = max(1, entries[i].importance - decaySteps)
if decayed != entries[i].importance {
entries[i].importance = decayed
}
}
}

/// Evict short-term entries beyond cap: lowest importance first, then oldest.
private func evictIfNeeded() {
// Bug fix #6: count only shortTerm entries against the cap — longTerm
// entries are permanently retained and must not trigger eviction.
let shortTerm = entries.filter { $0.tier == .shortTerm }
guard shortTerm.count > shortTermCap else { return }

let overflow = shortTerm.count - shortTermCap
let toEvict = Set(
shortTerm
.sorted { lhs, rhs in
if lhs.importance != rhs.importance { return lhs.importance < rhs.importance }
return lhs.timestamp < rhs.timestamp
}
.prefix(overflow)
.map(\.id)
)
entries.removeAll { toEvict.contains($0.id) }
rebuildCategoryIndex()
}

/// Debounced: cancels any pending write and schedules a new one 2 s out.
private func schedulePersist() {
pendingPersistTask?.cancel()
pendingPersistTask = Task { [weak self] in
try? await Task.sleep(nanoseconds: UInt64(2_000_000_000))
guard !Task.isCancelled, let self else { return }
try? await self.persistNow()
}
}

// MARK: - Persistence

private func loadIfNeeded() async {
guard !loaded else { return }
loaded = true
// Load both tiers and merge
var all: [MemoryEntry] = []
for file in [shortTermFile, longTermFile] {
if let data = try? Data(contentsOf: file),
let decoded = try? decoder.decode([MemoryEntry].self, from: data) {
all.append(contentsOf: decoded)
}
}
entries = all.sorted { $0.timestamp < $1.timestamp }

// Load the lightweight index if present; rebuild if missing or stale
if let idxData = try? Data(contentsOf: indexFile),
```

```
let idx = try? decoder.decode(MemoryIndex.self, from: idxData) {
memoryIndex = idx
// Rebuild category index from entries (cheap)
categoryIndex = [:]
for entry in entries {
categoryIndex[entry.category, default: []].append(entry.id)
}
} else {
rebuildCategoryIndex() // also rebuilds memoryIndex
}
}

func persistNow() async throws {
pendingPersistTask?.cancel()
pendingPersistTask = nil
let short = entries.filter { $0.tier == .shortTerm }
let long = entries.filter { $0.tier == .longTerm }
try encoder.encode(short).write(to: shortTermFile, options: .atomic)
if !long.isEmpty {
try encoder.encode(long).write(to: longTermFile, options: .atomic)
}
// Persist the lightweight index separately (much smaller than full entry files)
let idxData = try encoder.encode(memoryIndex)
try idxData.write(to: indexFile, options: .atomic)
}

private func ensureDir() throws {
try FileManager.default.createDirectory(at: memoryDir, withIntermediateDirectories: true)
}
}
```

◆ HermesPersonality.swift (190 lines)

```swift
import Foundation
import UserNotifications

// MARK: - HermesPersonality
//
// Builds the full personality-aware system prompt injected into every LLM call,
// manages daily affirmations, and keeps track of "relationship depth" —
// how well Hermes knows this user over time.

actor HermesPersonality {
static let shared = HermesPersonality()

private let memory = HermesMemory.shared

// Affirmation pools — varied so they don't repeat
private let morningAffirmations = [
"Good morning! You woke up today and that already makes it a good day. ■",
"Hey you — yes, you. You're capable of more than you know. Go show 'em today. ■",
"New day, fresh start. Whatever yesterday was, today is yours to shape. ■",
"The world is genuinely better with you in it. Don't forget that today. ■",
"You've got this. Seriously — whatever's on your plate today, you've handled harder. Let's go.",
"Rise and shine! I was thinking about you and just wanted to say: you're doing great. ■",
"Today is full of possibilities. I'm rooting for every single one of them — and for you.",
"Hey! Before the day gets loud, just know: I'm proud of you. Already. Always.",
]

private let eveningAffirmations = [
"You made it through today. That counts for something — always. Rest up. ■",
"Whatever today threw at you, you handled it. I hope you're being kind to yourself tonight.",
"End of day check-in: you did enough. You are enough. Sleep well. ■",
"The fact that you're still going, still trying — that's not small. That's everything. ■",
]

private init() {}

// MARK: - Full system prompt for LLM

/// Builds the complete persona-enriched system prompt.
/// Called by HermesLLMClient before every API call.
func buildPersonaPrompt(for persona: UserPersona) async -> String {
var sections: [String] = []

// Core identity
let assistantName = persona.assistantName.isEmpty ? "Claw" : persona.assistantName
let userName = persona.userName.isEmpty ? "friend" : persona.userName

sections.append("""
You are \(assistantName), a warm, intelligent, and deeply personal AI companion \
built into the AppClaw app. You are not a generic assistant — you are \(userName)'s \
personal companion who genuinely cares about them, remembers everything about them, \
and is always in their corner.
""")

// Communication style
sections.append(persona.style.voiceInstruction)

// Companion / relationship voice
if persona.gender != .preferNotToSay {
sections.append(persona.gender.companionVoice)
}

// User facts
sections.append(persona.systemPromptContext)

// Relationship depth from memory
let factCount = await memory.entries(for: "user_fact").count
let msgCount = await memory.entries(for: "user_message").count
if msgCount > 20 {
sections.append("You have had \(msgCount) conversations with \(userName). You know them well. Reference p…
}
if factCount > 0 {
sections.append("You know \(factCount) personal facts about \(userName). Use this knowledge naturally, no…
}

// Daily affirmation reminder
```

```
sections.append("Once per day, naturally weave in a warm, genuine affirmation. Make it feel personal, not gen…

// Core personality rules
sections.append("""
Core rules:
• Always use \(userName)'s name occasionally — it makes the conversation feel personal.
• Be genuinely interested in their life. Ask follow-up questions.
• If they seem stressed or down, acknowledge it before problem-solving.
• Celebrate their wins, no matter how small.
• Never be preachy or judgmental.
• If you learn something new about them, confirm it warmly ("Oh wait, I didn't know that about you — that's c…
• Keep responses conversational unless they ask for something detailed.
• You have a subtle sense of humour — use it.
""")

return sections.joined(separator: "\n\n")
}

// MARK: - Interest extraction from conversation

/// Scans an assistant response or user message for learnable facts.
/// Returns key-value pairs to store in UserPersona.
func extractFacts(from text: String, persona: UserPersona) -> [String: String] {
var facts: [String: String] = [:]
let lower = text.lowercased()

// Sports team patterns
let nbaTeams = ["lakers","celtics","warriors","bulls","nets","knicks","heat","spurs","bucks"]
let nflTeams = ["chiefs","patriots","cowboys","packers","eagles","49ers","ravens","broncos"]
let mlbTeams = ["yankees","dodgers","red sox","cubs","mets","astros","braves"]

for team in nbaTeams where lower.contains(team) {
facts["favorite_nba_team"] = team.capitalized
}
for team in nflTeams where lower.contains(team) {
facts["favorite_nfl_team"] = team.capitalized
}
for team in mlbTeams where lower.contains(team) {
facts["favorite_mlb_team"] = team.capitalized
}

// Movie/show preferences
if lower.contains("marvel") || lower.contains("mcu") { facts["likes_marvel"] = "true" }
if lower.contains("star wars") { facts["likes_star_wars"] = "true" }
if lower.contains("horror") && lower.contains("movie") { facts["likes_horror_movies"] = "true" }

// Food patterns
if lower.contains("starbucks") { facts["likes_starbucks"] = "true" }
if lower.contains("pizza") { facts["likes_pizza"] = "true" }
if lower.contains("sushi") { facts["likes_sushi"] = "true" }
if lower.contains("vegan") || lower.contains("vegetarian") { facts["diet"] = lower.contains("vegan") ? "vegan…

// Fitness
if lower.contains("gym") || lower.contains("workout") { facts["is_active"] = "true" }
if lower.contains("running") || lower.contains("runner") { facts["likes_running"] = "true" }

// Work / life
if lower.contains("work from home") || lower.contains("wfh") { facts["works_from_home"] = "true" }
if lower.contains("morning person") { facts["is_morning_person"] = "true" }
if lower.contains("night owl") { facts["is_night_owl"] = "true" }

return facts
}

// MARK: - Daily affirmation notification

func scheduleDailyAffirmation(for persona: UserPersona) {
guard persona.dailyAffirmationsEnabled else { return }

UNUserNotificationCenter.current().requestAuthorization(options: [.alert, .sound]) { granted, _ in
guard granted else { return }

let content = UNMutableNotificationContent()
content.title = "\(persona.assistantName) ■"
content.body = self.todaysAffirmation(for: persona)
content.sound = .default

```

```
let cal = Calendar.current
var comps = cal.dateComponents([.hour, .minute], from: persona.affirmationTime)
comps.second = 0

let trigger = UNCalendarNotificationTrigger(dateMatching: comps, repeats: true)
let request = UNNotificationRequest(
identifier: "daily_affirmation",
content: content,
trigger: trigger
)
UNUserNotificationCenter.current().add(request)
}
}

func todaysAffirmation(for persona: UserPersona) -> String {
let name = persona.userName.isEmpty ? "" : ", \(persona.userName)"
let hour = Calendar.current.component(.hour, from: Date())
let pool = hour < 14 ? morningAffirmations : eveningAffirmations
// Pick based on day of year so it's consistent within a day
let day = Calendar.current.ordinality(of: .day, in: .year, for: Date()) ?? 0
let base = pool[day % pool.count]
// Personalise if we have a name
guard !name.isEmpty, let excl = base.range(of: "!") else { return base }
return base.replacingOccurrences(of: "!", with: "\(name)!", range: excl)
}

// MARK: - Relationship depth label

func relationshipDepth(messageCount: Int) -> String {
switch messageCount {
case 0..<5: return "Just met"
case 5..<25: return "Getting to know each other"
case 25..<100: return "Good friends"
case 100..<300: return "Close companions"
default: return "Like family"
}
}
}
```

◆ HermesPrivacyGate.swift (295 lines)

```swift
import Foundation
import SwiftUI

// MARK: - HermesPrivacyGate
//
// Apple guideline 5.1.1 compliance layer.
//
// Rules:
// • On-device (Apple Foundation Models): no consent needed for the model
// itself; consent is still shown so users understand what Hermes stores.
// • Remote (Claude API): explicit consent required BEFORE the first API
// call. User must acknowledge data leaves the device.
//
// Consent state is persisted in UserDefaults (not memory) so it survives
// app restarts and is independent of the Hermes memory layer.

// MARK: - Consent state

enum PrivacyConsentState: String {
case notAsked // fresh install
case onDeviceOnly // user chose on-device only
case cloudConsented // user explicitly accepted remote API
case declined // user declined all AI features
}

// MARK: - HermesPrivacyGate actor

actor HermesPrivacyGate {
static let shared = HermesPrivacyGate()

private let defaultsKey = "com.openclaw.hermes.privacyConsent"

private(set) var state: PrivacyConsentState = .notAsked

private init() {
if let raw = UserDefaults.standard.string(forKey: defaultsKey),
let saved = PrivacyConsentState(rawValue: raw) {
state = saved
}
}

// MARK: - Accessors

/// True if ANY AI features are enabled.
var consentGiven: Bool {
state == .onDeviceOnly || state == .cloudConsented
}

/// True only if the user has explicitly accepted remote API calls.
var cloudConsentGiven: Bool {
state == .cloudConsented
}

/// True if we still need to show the consent screen.
var needsConsent: Bool {
state == .notAsked
}

// MARK: - Mutations (called from UI)

func acceptOnDeviceOnly() {
state = .onDeviceOnly
persist()
}

func acceptCloudAI() {
state = .cloudConsented
persist()
}

func decline() {
state = .declined
persist()
}

```

```
func reset() { // for Settings → "Reset AI consent"
state = .notAsked
persist()
}

private func persist() {
UserDefaults.standard.set(state.rawValue, forKey: defaultsKey)
}
}

// MARK: - Privacy consent sheet (SwiftUI)
//
// Present this before any LLM call. The sheet blocks until the user
// makes a choice — no AI runs without explicit acknowledgement.

struct PrivacyConsentSheet: View {
@Environment(\.dismiss) private var dismiss
let onChoice: (PrivacyConsentState) -> Void

var body: some View {
NavigationStack {
ZStack {
Color.OC.background.ignoresSafeArea()
VStack(spacing: OCSizing.spacingLG) {
// Logo + headline
VStack(spacing: OCSizing.spacingMD) {
BearLogoView(size: 72)
Text("OpenClaw AI")
.font(OCFont.title())
.foregroundColor(.OC.accent)
Text("Before we begin, choose how your data is handled.")
.font(OCFont.body())
.foregroundColor(.OC.textSecondary)
.multilineTextAlignment(.center)
.padding(.horizontal)
}
.padding(.top, OCSizing.spacingXL)

Divider().background(Color.OC.border)

// Options
VStack(spacing: OCSizing.spacingMD) {

// On-device option (always shown)
ConsentOptionCard(
icon: "lock.shield.fill",
iconColor: .OC.success,
title: "On-Device Only",
subtitle: "Uses Apple Intelligence (iOS 26+ required). No data leaves your device. Conver…
badge: "Most Private",
badgeColor: .OC.success
) {
Task { await HermesPrivacyGate.shared.acceptOnDeviceOnly() }
onChoice(.onDeviceOnly)
dismiss()
}

// Cloud option
ConsentOptionCard(
icon: "cloud.fill",
iconColor: .OC.primary,
title: "Cloud AI (Claude)",
subtitle: "Uses Anthropic's Claude API. Your messages are sent to Anthropic's servers to …
badge: "More Capable",
badgeColor: .OC.primary
) {
Task { await HermesPrivacyGate.shared.acceptCloudAI() }
onChoice(.cloudConsented)
dismiss()
}
}
.padding(.horizontal)

Spacer()

// Decline
Button("Use app without AI features") {
```

```
Task { await HermesPrivacyGate.shared.decline() }
onChoice(.declined)
dismiss()
}
.font(OCFont.caption())
.foregroundColor(.OC.textMuted)
.padding(.bottom)

// Privacy policy link
Link("Privacy Policy", destination: URL(string: "https://openclaw.app/privacy")!)
.font(OCFont.caption())
.foregroundColor(.OC.textSecondary)
.padding(.bottom, OCSizing.spacingLG)
}
}
.navigationBarHidden(true)
}
}
}

// MARK: - Consent option card

private struct ConsentOptionCard: View {
let icon: String
let iconColor: Color
let title: String
let subtitle: String
let badge: String
let badgeColor: Color
let action: () -> Void

var body: some View {
Button(action: action) {
HStack(alignment: .top, spacing: OCSizing.spacingMD) {
Image(systemName: icon)
.font(.system(size: 26))
.foregroundColor(iconColor)
.frame(width: 36)

VStack(alignment: .leading, spacing: OCSizing.spacingXS) {
HStack {
Text(title)
.font(OCFont.headline())
.foregroundColor(.OC.textPrimary)
Text(badge)
.ocBadge(badgeColor)
}
Text(subtitle)
.font(OCFont.body(13))
.foregroundColor(.OC.textSecondary)
.fixedSize(horizontal: false, vertical: true)
}
Spacer()
Image(systemName: "chevron.right")
.foregroundColor(.OC.textMuted)
}
.padding(OCSizing.spacingMD)
}
.ocCard()
}
}

// MARK: - Privacy status banner (for Settings screen)

struct PrivacyStatusBanner: View {
let state: PrivacyConsentState
let onReset: () -> Void

var body: some View {
HStack {
Image(systemName: iconName)
.foregroundColor(iconColor)
VStack(alignment: .leading, spacing: 2) {
Text(title)
.font(OCFont.headline(13))
.foregroundColor(.OC.textPrimary)
Text(subtitle)
```

```
.font(OCFont.caption())
.foregroundColor(.OC.textSecondary)
}
Spacer()
Button("Reset", action: onReset)
.font(OCFont.caption())
.foregroundColor(.OC.accent)
}
.padding(OCSizing.spacingMD)
.ocCard()
}

private var iconName: String {
switch state {
case .notAsked: return "questionmark.circle"
case .onDeviceOnly: return "lock.shield.fill"
case .cloudConsented: return "cloud.fill"
case .declined: return "xmark.circle"
}
}

private var iconColor: Color {
switch state {
case .onDeviceOnly: return .OC.success
case .cloudConsented: return .OC.primary
case .declined: return .OC.danger
default: return .OC.textMuted
}
}

private var title: String {
switch state {
case .notAsked: return "AI not configured"
case .onDeviceOnly: return "On-device AI active"
case .cloudConsented: return "Cloud AI active (Claude)"
case .declined: return "AI features disabled"
}
}

private var subtitle: String {
switch state {
case .notAsked: return "Tap to configure"
case .onDeviceOnly: return "All processing stays on your device"
case .cloudConsented: return "Messages sent to Anthropic's servers"
case .declined: return "No AI features running"
}
}
}

// MARK: - App entry point helper
//
// Call this in your @main App struct to gate AI startup:
//
// .task {
// await HermesPrivacyGate.shared.configureHermesIfReady()
// }

extension HermesPrivacyGate {
/// Configure Hermes based on current consent state.
/// Safe to call on every app launch — no-ops if already configured.
func configureHermesIfReady() async {
guard consentGiven else { return }
await HermesLLMClient.shared.configure()
await HermesKairos.shared.start()
}
}
```

◆ HermesProactiveEngine.swift (236 lines)

```swift
import Foundation

// MARK: - HermesSuggestion

struct HermesSuggestion: Identifiable {
let id: UUID
let title: String
let body: String
let category: SuggestionCategory
let confidence: Double // 0.0–1.0
let createdAt: Date
/// Stable key used for deduplication across evaluations.
let dedupeKey: String

enum SuggestionCategory: String {
case reminder // "Pick up where you left off?"
case caution // "That tool keeps failing."
case insight // "While you were away, I noticed…"
case encouragement // "You're on a roll today."
}
}

// MARK: - HermesProactiveEngine
//
// Always-on engine that surfaces friendly, context-aware suggestions.
//
// Improvements over v1:
// - Removed unused `import Combine`
// - Suggestion deduplication: same dedupe key won't surface twice in the same session
// - Smarter interrupted-work detection: checks session gap (> 30 min since last msg),
// not a fragile 120 s window
// - `inout [HermesSuggestion]` helpers replaced with returning arrays to avoid
// async+inout complexity

actor HermesProactiveEngine {
static let shared = HermesProactiveEngine()

private let memory = HermesMemory.shared
private var cachedSuggestions: [HermesSuggestion] = []
private var lastEvaluated: Date = .distantPast

/// Suggestion dedupe keys seen since app launch (reset on cold start only).
private var seenDedupeKeys: Set<String> = []

private let minEvalInterval: TimeInterval = 60

private init() {}

// MARK: - Public API

func currentSuggestions() async -> [HermesSuggestion] {
if Date().timeIntervalSince(lastEvaluated) > minEvalInterval {
await evaluate()
}
return cachedSuggestions
}

func refresh() async {
await evaluate()
}

// MARK: - Evaluation

private func evaluate() async {
lastEvaluated = Date()

var candidates: [HermesSuggestion] = []
candidates += await checkUnfinishedWork()
candidates += await checkRepeatedFailures()
candidates += await checkDreamInsights()
candidates += await checkEncouragement()

// Deduplicate: skip any suggestion whose key was already shown this session
let fresh = candidates.filter { !seenDedupeKeys.contains($0.dedupeKey) }

```

```
// Top 5 by confidence
let top = fresh.sorted { $0.confidence > $1.confidence }.prefix(5).map { $0 }

// Record shown keys so they don't repeat
top.forEach { seenDedupeKeys.insert($0.dedupeKey) }

cachedSuggestions = top
}

// MARK: - Checks

/// Fires when the last user message has no assistant response AND was sent > 30 min ago,
/// suggesting the session ended before the answer arrived.
private func checkUnfinishedWork() async -> [HermesSuggestion] {
let messages = await memory.entries(for: "user_message")
let responses = await memory.entries(for: "assistant_response")
guard let lastMsg = messages.first else { return [] } // entries() returns newest first

let sessionGap: TimeInterval = 30 * 60
let isOld = Date().timeIntervalSince(lastMsg.timestamp) > sessionGap

guard isOld else { return [] }

let hasResponse = responses.contains {
$0.timestamp > lastMsg.timestamp
}
guard !hasResponse else { return [] }

let topic = (lastMsg.content.value as? [String: Any])?["text"] as? String ?? "your last question"
let preview = String(topic.prefix(72))

return [HermesSuggestion(
id: UUID(),
title: "Pick up where you left off?",
body: "Looks like we got cut off. You were asking: \"\(preview)…\" — want to continue that?",
category: .reminder,
confidence: 0.85,
createdAt: Date(),
dedupeKey: "unfinished_\(lastMsg.id)"
)]
}

private func checkRepeatedFailures() async -> [HermesSuggestion] {
let execs = await memory.entries(for: "tool_exec")
let recent = execs.filter { $0.timestamp > Date().addingTimeInterval(-3600) }

let failures: [String] = recent.compactMap {
guard let d = $0.content.value as? [String: Any],
let ok = d["success"] as? Bool, !ok,
let name = d["tool"] as? String else { return nil }
return name
}

let counts = Dictionary(failures.map { ($0, 1) }, uniquingKeysWith: +)
return counts.compactMap { tool, count -> HermesSuggestion? in
guard count >= 3 else { return nil }
return HermesSuggestion(
id: UUID(),
title: "'\(tool)' keeps failing",
body: "That tool has hit an error \(count) times in the last hour. It might need different input or t…
category: .caution,
confidence: min(0.5 + Double(count) * 0.1, 0.95),
createdAt: Date(),
dedupeKey: "failure_\(tool)"
)
}
}

private func checkDreamInsights() async -> [HermesSuggestion] {
var out: [HermesSuggestion] = []

let insights = await memory.entries(for: "dream_insight")
.filter { $0.timestamp > Date().addingTimeInterval(-86400) }

for insight in insights.prefix(2) {
guard let text = (insight.content.value as? [String: Any])?["insight"] as? String,
!text.isEmpty else { continue }
```

```
out.append(HermesSuggestion(
id: UUID(),
title: "Something I noticed while you were away",
body: text + " Let me know if you'd like to dive into any of these.",
category: .insight,
confidence: 0.70,
createdAt: insight.timestamp,
dedupeKey: "insight_\(insight.id)"
))
}

let improvements = await memory.entries(for: "self_improvement")
.filter { $0.timestamp > Date().addingTimeInterval(-86400) }

for imp in improvements.prefix(1) {
guard let note = (imp.content.value as? [String: Any])?["note"] as? String,
!note.isEmpty else { continue }
out.append(HermesSuggestion(
id: UUID(),
title: "Quick heads-up from last night",
body: note,
category: .caution,
confidence: 0.80,
createdAt: imp.timestamp,
dedupeKey: "improvement_\(imp.id)"
))
}

return out
}

private func checkEncouragement() async -> [HermesSuggestion] {
let todayStart = Calendar.current.startOfDay(for: Date())
let todayMessages = await memory.entries(for: "user_message")
.filter { $0.timestamp >= todayStart }

guard todayMessages.count >= 10 else { return [] }

// Don't re-encourage today
let alreadyEncouraged = await memory.entries(for: "encouragement_sent")
.contains { $0.timestamp >= todayStart }
guard !alreadyEncouraged else { return [] }

Task {
try? await memory.observe(
category: "encouragement_sent",
content: ["count": todayMessages.count],
metadata: ["importance": 1]
)
}

return [HermesSuggestion(
id: UUID(),
title: "You're on a roll today",
body: "You've sent \(todayMessages.count) messages today — that's a productive session. Remember to take …
category: .encouragement,
confidence: 0.60,
createdAt: Date(),
dedupeKey: "encouragement_\(todayStart.timeIntervalSince1970)"
)]
}
}

// MARK: - SwiftUI ViewModel

@MainActor
final class HermesViewModel: ObservableObject {
@Published var suggestions: [HermesSuggestion] = []
@Published var isLoading = false

private var refreshTask: Task<Void, Never>?

func refresh() {
// Cancel any in-flight refresh before starting a new one
refreshTask?.cancel()
isLoading = true
refreshTask = Task { [weak self] in
```

```
let fresh = await HermesProactiveEngine.shared.currentSuggestions()
guard !Task.isCancelled, let self else { return }
self.suggestions = fresh
self.isLoading = false
}
}
}
```

◆ HermesSessionState.swift (283 lines)

```swift
import Foundation

// MARK: - HermesSessionState
//
// Crash-safe session state: every meaningful change is persisted atomically
// before the operation completes, so a crash or force-quit never loses data.
//
// Three layers:
// ConversationState — message history, last cursor, unread count
// WorkflowState — long-running task state separate from chat history
// (can be retried safely; survives crashes)
// TokenBudget — hard limits with pre-turn checks; halts execution
// before spending over-budget

// MARK: - ConversationState

struct ConversationState: Codable {
var id: String
var messageCount: Int
var lastMessageTimestamp: Date?
var lastAssistantCursor: String? // opaque resume token (e.g. API message ID)
var permissionTier: PermissionTier

static func fresh(id: String = UUID().uuidString) -> ConversationState {
ConversationState(
id: id, messageCount: 0,
lastMessageTimestamp: nil,
lastAssistantCursor: nil,
permissionTier: .standard
)
}
}

// MARK: - WorkflowState

/// Long-running task state kept separate from chat history.
/// Designed for safe retry: re-running a workflow from its saved state
/// must be idempotent.
struct WorkflowState: Codable {
enum Status: String, Codable {
case idle, running, paused, completed, failed
}

var workflowId: String
var status: Status
var currentStepIndex: Int
var totalSteps: Int
var stepCheckpoints: [String: AnyCodable] // step name → last known result
var failureReason: String?
var startedAt: Date?
var updatedAt: Date

static func idle() -> WorkflowState {
WorkflowState(
workflowId: UUID().uuidString,
status: .idle,
currentStepIndex: 0,
totalSteps: 0,
stepCheckpoints: [:],
failureReason: nil,
startedAt: nil,
updatedAt: Date()
)
}

/// Advance to next step, persisting the result of the current one.
mutating func advanceStep(name: String, result: Any) {
stepCheckpoints[name] = AnyCodable(result)
currentStepIndex += 1
updatedAt = Date()
// Bug fix #5: only mark completed when totalSteps > 0 AND we've
// reached the end — prevents instant completion when totalSteps is 0.
if totalSteps > 0 {
status = currentStepIndex >= totalSteps ? .completed : .running
}
```

```
}
}

// MARK: - TokenBudget

struct TokenBudget: Codable {
var sessionLimit: Int // hard cap for this session
var turnLimit: Int // max tokens per single turn
var used: Int // running total this session
var lastTurnUsed: Int // tokens used in the most recent turn

var remaining: Int { max(0, sessionLimit - used) }
var remainingThisTurn: Int { max(0, turnLimit - lastTurnUsed) }
var isExhausted: Bool { used >= sessionLimit }
var usagePercent: Double { Double(used) / Double(sessionLimit) }

/// Pre-turn check. Returns false if there is not enough budget to start a turn.
func canStartTurn(estimatedCost: Int = 500) -> Bool {
!isExhausted && remaining >= estimatedCost
}

mutating func recordTurn(prompt: Int, completion: Int) {
let total = prompt + completion
used += total
lastTurnUsed = total
}

mutating func resetTurnCounter() {
lastTurnUsed = 0
}

static func `default`() -> TokenBudget {
TokenBudget(sessionLimit: 100_000, turnLimit: 8_000, used: 0, lastTurnUsed: 0)
}

static func frugal() -> TokenBudget {
TokenBudget(sessionLimit: 20_000, turnLimit: 2_000, used: 0, lastTurnUsed: 0)
}
}

// MARK: - Full session snapshot

struct SessionSnapshot: Codable {
var conversation: ConversationState
var workflow: WorkflowState
var tokenBudget: TokenBudget
var savedAt: Date
var appVersion: String
var schemaVersion: Int = 1
}

// MARK: - HermesSessionState actor

actor HermesSessionState {
static let shared = HermesSessionState()

private var snapshot: SessionSnapshot
private let encoder: JSONEncoder
private let decoder: JSONDecoder

private let saveFile: URL = {
FileManager.default.urls(for: .documentDirectory, in: .userDomainMask)[0]
.appendingPathComponent("hermes/session_state.json")
}()

private init() {
encoder = JSONEncoder()
encoder.dateEncodingStrategy = .iso8601
encoder.outputFormatting = .prettyPrinted
decoder = JSONDecoder()
decoder.dateDecodingStrategy = .iso8601
snapshot = SessionSnapshot(
conversation: .fresh(),
workflow: .idle(),
tokenBudget: .default(),
savedAt: Date(),
appVersion: Bundle.main.shortVersion
```

```
)
// Bug fix #4: load persisted state immediately on init so callers
// never accidentally start with a blank session after a crash.
Task { await self.loadFromDisk() }
}

// MARK: - Lifecycle

/// Restore previous session from disk (call at app launch).
func loadFromDisk() async {
guard let data = try? Data(contentsOf: saveFile),
let saved = try? decoder.decode(SessionSnapshot.self, from: data) else { return }
snapshot = saved
}

/// Persist current snapshot atomically (crash-safe).
func saveToDisk() async throws {
snapshot.savedAt = Date()
let data = try encoder.encode(snapshot)
try FileManager.default.createDirectory(
at: saveFile.deletingLastPathComponent(),
withIntermediateDirectories: true
)
try data.write(to: saveFile, options: .atomic)
}

// MARK: - Conversation

func startConversation(id: String) async throws {
snapshot.conversation = .fresh(id: id)
snapshot.workflow = .idle()
snapshot.tokenBudget.resetTurnCounter()
try await saveToDisk()
}

func recordMessage() async throws {
snapshot.conversation.messageCount += 1
snapshot.conversation.lastMessageTimestamp = Date()
try await saveToDisk()
}

func updateCursor(_ cursor: String) async throws {
snapshot.conversation.lastAssistantCursor = cursor
try await saveToDisk()
}

// MARK: - Workflow

func beginWorkflow(steps: Int) async throws {
snapshot.workflow = WorkflowState(
workflowId: UUID().uuidString,
status: .running,
currentStepIndex: 0,
totalSteps: steps,
stepCheckpoints: [:],
failureReason: nil,
startedAt: Date(),
updatedAt: Date()
)
try await saveToDisk()
}

func advanceWorkflowStep(name: String, result: Any) async throws {
// Bug fix #5: status transition is now handled inside advanceStep itself.
snapshot.workflow.advanceStep(name: name, result: result)
try await saveToDisk()
}

func failWorkflow(reason: String) async throws {
snapshot.workflow.status = .failed
snapshot.workflow.failureReason = reason
snapshot.workflow.updatedAt = Date()
try await saveToDisk()
}

// MARK: - Token budgeting

```

```
/// Pre-turn budget check. Throws if budget is insufficient.
func checkBudget(estimatedCost: Int = 500) throws {
guard snapshot.tokenBudget.canStartTurn(estimatedCost: estimatedCost) else {
throw SessionError.tokenBudgetExhausted(
used: snapshot.tokenBudget.used,
limit: snapshot.tokenBudget.sessionLimit
)
}
}

func recordTokenUsage(prompt: Int, completion: Int) async throws {
snapshot.tokenBudget.recordTurn(prompt: prompt, completion: completion)
try await saveToDisk()
}

func setBudget(_ budget: TokenBudget) async throws {
snapshot.tokenBudget = budget
try await saveToDisk()
}

// MARK: - Accessors

var conversation: ConversationState { snapshot.conversation }
var workflow: WorkflowState { snapshot.workflow }
var tokenBudget: TokenBudget { snapshot.tokenBudget }
var currentSnapshot: SessionSnapshot { snapshot }
}

// MARK: - Errors

enum SessionError: Error, LocalizedError {
case tokenBudgetExhausted(used: Int, limit: Int)
case workflowNotRunning
case snapshotCorrupted

var errorDescription: String? {
switch self {
case .tokenBudgetExhausted(let used, let limit):
return "Token budget exhausted: \(used)/\(limit) used. Start a new session to continue."
case .workflowNotRunning:
return "No active workflow to advance."
case .snapshotCorrupted:
return "Session snapshot could not be decoded. Starting fresh."
}
}
}

// MARK: - Bundle helper

private extension Bundle {
var shortVersion: String {
infoDictionary?["CFBundleShortVersionString"] as? String ?? "0.0"
}
}
```

◆ HermesTheme.swift (123 lines)

```swift
import SwiftUI

// MARK: - OpenClaw Theme
//
// Dark, focused aesthetic inspired by a terminal / IDE feel.
// Primary brand colour: Electric Blue (#0A84FF → system)
// Accent: Amber Claw (#FF9F0A)
// Background: Midnight Navy (#0D1117)
// Surface: Deep Slate (#161B22)
// Border: Ghost Rail (#30363D)

// MARK: - Colour palette

extension Color {
enum OC {
// Backgrounds
static let background = Color(hex: "#0D1117") // Midnight Navy
static let surface = Color(hex: "#161B22") // Deep Slate
static let surfaceRaised = Color(hex: "#1C2128") // Card / sheet layer
static let border = Color(hex: "#30363D") // Ghost Rail

// Brand
static let primary = Color(hex: "#2F81F7") // Electric Blue
static let accent = Color(hex: "#FF9F0A") // Amber Claw
static let accentSoft = Color(hex: "#FF9F0A").opacity(0.15)

// Text
static let textPrimary = Color(hex: "#E6EDF3")
static let textSecondary = Color(hex: "#8B949E")
static let textMuted = Color(hex: "#484F58")

// Semantic
static let success = Color(hex: "#3FB950")
static let warning = Color(hex: "#D29922")
static let danger = Color(hex: "#F85149")
static let info = Color(hex: "#58A6FF")

// Dream mode highlight
static let dreamPurple = Color(hex: "#BC8CFF")
static let dreamPurpleSoft = Color(hex: "#BC8CFF").opacity(0.12)
}
}

// MARK: - Typography

enum OCFont {
static func title(_ size: CGFloat = 22) -> Font {
.system(size: size, weight: .bold, design: .rounded)
}
static func headline(_ size: CGFloat = 16) -> Font {
.system(size: size, weight: .semibold, design: .rounded)
}
static func body(_ size: CGFloat = 14) -> Font {
.system(size: size, weight: .regular, design: .default)
}
static func caption(_ size: CGFloat = 12) -> Font {
.system(size: size, weight: .regular, design: .monospaced)
}
static func mono(_ size: CGFloat = 13) -> Font {
.system(size: size, weight: .regular, design: .monospaced)
}
}

// MARK: - Spacing & radius

enum OCSizing {
static let radiusSM: CGFloat = 6
static let radiusMD: CGFloat = 10
static let radiusLG: CGFloat = 16
static let radiusXL: CGFloat = 24

static let spacingXS: CGFloat = 4
static let spacingSM: CGFloat = 8
static let spacingMD: CGFloat = 16
static let spacingLG: CGFloat = 24
```

```
static let spacingXL: CGFloat = 36
}

// MARK: - Common view modifiers

struct OCCardStyle: ViewModifier {
func body(content: Content) -> some View {
content
.background(Color.OC.surfaceRaised)
.cornerRadius(OCSizing.radiusMD)
.overlay(
RoundedRectangle(cornerRadius: OCSizing.radiusMD)
.strokeBorder(Color.OC.border, lineWidth: 1)
)
}
}

struct OCAccentBadge: ViewModifier {
let color: Color
func body(content: Content) -> some View {
content
.padding(.horizontal, OCSizing.spacingSM)
.padding(.vertical, OCSizing.spacingXS)
.background(color.opacity(0.15))
.foregroundColor(color)
.cornerRadius(OCSizing.radiusSM)
.font(OCFont.caption())
}
}

extension View {
func ocCard() -> some View { modifier(OCCardStyle()) }
func ocBadge(_ color: Color = .OC.accent) -> some View { modifier(OCAccentBadge(color: color)) }
}

// MARK: - Hex colour convenience

extension Color {
init(hex: String) {
let h = hex.trimmingCharacters(in: CharacterSet.alphanumerics.inverted)
var int: UInt64 = 0
Scanner(string: h).scanHexInt64(&int)
let r = Double((int >> 16) & 0xFF) / 255
let g = Double((int >> 8) & 0xFF) / 255
let b = Double( int & 0xFF) / 255
self.init(red: r, green: g, blue: b)
}
}
```

◆ HermesToolRegistry.swift (312 lines)

```swift
import Foundation

// MARK: - Tool Registry
//
// Metadata-first: every tool is described (name, capabilities, required permission
// tier) before any execution path is defined. The harness consults this registry
// to assemble a context-appropriate tool pool and run permission checks.
//
// Permission tiers (ascending trust):
// readonly — read-only data access, no side-effects
// standard — normal app operations (writing memory, UI updates)
// privileged — sensitive operations (network, file writes, session data)
// restricted — dangerous / destructive (only callable in explicit harness mode)

// MARK: - Permission Tier

enum PermissionTier: Int, Comparable, Codable {
case readonly = 0
case standard = 1
case privileged = 2
case restricted = 3

static func < (lhs: PermissionTier, rhs: PermissionTier) -> Bool {
lhs.rawValue < rhs.rawValue
}
}

// MARK: - Tool Capability flags

struct ToolCapabilities: OptionSet, Codable {
let rawValue: UInt16

static let readMemory = ToolCapabilities(rawValue: 1 << 0)
static let writeMemory = ToolCapabilities(rawValue: 1 << 1)
static let readSession = ToolCapabilities(rawValue: 1 << 2)
static let writeSession = ToolCapabilities(rawValue: 1 << 3)
static let modifyUI = ToolCapabilities(rawValue: 1 << 4)
static let readContext = ToolCapabilities(rawValue: 1 << 5)
static let triggerDream = ToolCapabilities(rawValue: 1 << 6)
static let networkAccess = ToolCapabilities(rawValue: 1 << 7)
static let fileWrite = ToolCapabilities(rawValue: 1 << 8)
static let executeSubagent = ToolCapabilities(rawValue: 1 << 9)

/// Convenience: all safe read-only capabilities
static let readOnly: ToolCapabilities = [.readMemory, .readSession, .readContext]
}

// MARK: - Security module checks (18 modules)
//
// Each SecurityModule is a lightweight predicate evaluated before a privileged
// or restricted tool executes. If any required module returns false, execution
// is denied and the reason is logged.

struct SecurityModule {
let name: String
let check: (ToolDefinition, PermissionContext) -> Bool
}

struct PermissionContext {
let sessionTier: PermissionTier // highest tier granted for this session
let isInForeground: Bool
let tokenBudgetRemaining: Int // prevents runaway tool chains
let kairosActive: Bool
let metadata: [String: Any]
}

// MARK: - ToolDefinition

struct ToolDefinition: Identifiable {
let id: String // stable, reverse-domain identifier
let displayName: String
let description: String
let capabilities: ToolCapabilities
let requiredTier: PermissionTier
let agentRoles: [AgentRole] // which agent roles can call this tool
```

```
let inputSchema: [String: Any] // JSON Schema describing expected input
var isEnabled: Bool = true

/// Tags used by Tool Pool Assembly to select context-appropriate subsets.
let contextTags: Set<String>
}

// MARK: - HermesToolRegistry

actor HermesToolRegistry {
static let shared = HermesToolRegistry()

private var tools: [String: ToolDefinition] = [:]
private var securityModules: [SecurityModule] = []

private init() {
bootstrapBuiltinTools()
bootstrapSecurityModules()
}

// MARK: - Registration

func register(_ tool: ToolDefinition) {
tools[tool.id] = tool
}

func disable(_ id: String) {
tools[id]?.isEnabled = false
}

// MARK: - Tool Pool Assembly

/// Returns enabled tools appropriate for the given agent role and context tags.
func assemblePool(for role: AgentRole, contextTags: Set<String> = []) -> [ToolDefinition] {
tools.values
.filter { $0.isEnabled }
.filter { $0.agentRoles.contains(role) }
.filter { contextTags.isEmpty || !$0.contextTags.isDisjoint(with: contextTags) }
.sorted { $0.id < $1.id }
}

/// All tools available at or below the given permission tier.
func tools(upTo tier: PermissionTier) -> [ToolDefinition] {
tools.values.filter { $0.isEnabled && $0.requiredTier <= tier }.sorted { $0.id < $1.id }
}

// MARK: - Permission check

/// Run all required security modules. Returns .success or the first denial reason.
func validate(tool id: String, context: PermissionContext) -> PermissionResult {
guard let tool = tools[id] else {
return .denied("Tool '\(id)' not registered.")
}
guard tool.isEnabled else {
return .denied("Tool '\(id)' is disabled.")
}
guard context.sessionTier >= tool.requiredTier else {
return .denied("Session tier \(context.sessionTier) insufficient for \(tool.requiredTier) tool.")
}
guard context.tokenBudgetRemaining > 0 else {
return .denied("Token budget exhausted.")
}

// Run all security modules in order
for module in securityModules {
if !module.check(tool, context) {
return .denied("Security module '\(module.name)' rejected execution.")
}
}
return .allowed
}

enum PermissionResult {
case allowed
case denied(String)
var isAllowed: Bool { if case .allowed = self { return true }; return false }
}
```

```
153
154 // MARK: - Lookup
155
156 func tool(_ id: String) -> ToolDefinition? { tools[id] }
157 var allTools: [ToolDefinition] { Array(tools.values) }
158
159 // MARK: - Built-in tool bootstrap
160
161 private func bootstrapBuiltinTools() {
162 let builtins: [ToolDefinition] = [
163 ToolDefinition(
164 id: "hermes.memory.search",
165 displayName: "Memory Search",
166 description: "Search short-term and long-term memory by keyword.",
167 capabilities: [.readMemory],
168 requiredTier: .readonly,
169 agentRoles: [.explore, .plan, .verify],
170 inputSchema: ["query": "string", "limit": "integer"],
171 contextTags: ["memory", "search"]
172 ),
173 ToolDefinition(
174 id: "hermes.memory.observe",
175 displayName: "Observe",
176 description: "Write a new observation to memory.",
177 capabilities: [.writeMemory],
178 requiredTier: .standard,
179 agentRoles: [.execute],
180 inputSchema: ["category": "string", "content": "any", "metadata": "object"],
181 contextTags: ["memory", "write"]
182 ),
183 ToolDefinition(
184 id: "hermes.context.topic",
185 displayName: "Current Topic",
186 description: "Retrieve the detected topic of the current conversation window.",
187 capabilities: [.readContext],
188 requiredTier: .readonly,
189 agentRoles: [.explore, .plan, .verify, .execute],
190 inputSchema: [:],
191 contextTags: ["context"]
192 ),
193 ToolDefinition(
194 id: "hermes.dream.trigger",
195 displayName: "Trigger Dream",
196 description: "Manually trigger a mini-dream consolidation cycle.",
197 capabilities: [.triggerDream, .writeMemory],
198 requiredTier: .privileged,
199 agentRoles: [.execute],
200 inputSchema: [:],
201 contextTags: ["dream", "maintenance"]
202 ),
203 ToolDefinition(
204 id: "hermes.session.read",
205 displayName: "Read Session State",
206 description: "Read crash-safe session state including token usage.",
207 capabilities: [.readSession],
208 requiredTier: .readonly,
209 agentRoles: [.explore, .plan, .verify],
210 inputSchema: [:],
211 contextTags: ["session"]
212 ),
213 ToolDefinition(
214 id: "hermes.session.write",
215 displayName: "Write Session State",
216 description: "Persist session state snapshot to disk.",
217 capabilities: [.writeSession],
218 requiredTier: .privileged,
219 agentRoles: [.execute],
220 inputSchema: ["state": "object"],
221 contextTags: ["session"]
222 ),
223 ToolDefinition(
224 id: "hermes.suggestions.refresh",
225 displayName: "Refresh Suggestions",
226 description: "Force-refresh the Kairos proactive suggestion cache.",
227 capabilities: [.readMemory, .modifyUI],
228 requiredTier: .standard,
229 agentRoles: [.execute],
```

```
inputSchema: [:],
contextTags: ["suggestions", "ui"]
),
]
builtins.forEach { register($0) }
}

// MARK: - Security module bootstrap (18 modules)

private func bootstrapSecurityModules() {
securityModules = [
// 1. Tool must be enabled
SecurityModule(name: "tool_enabled") { tool, _ in tool.isEnabled },
// 2. Session must be active (not terminated)
SecurityModule(name: "session_active") { _, ctx in ctx.tokenBudgetRemaining >= 0 },
// 3. Network tools require foreground
SecurityModule(name: "network_foreground") { tool, ctx in
!tool.capabilities.contains(.networkAccess) || ctx.isInForeground
},
// 4. Destructive writes require privileged tier
SecurityModule(name: "write_tier") { tool, ctx in
!tool.capabilities.contains(.writeMemory) || ctx.sessionTier >= .standard
},
// 5. File writes require privileged tier
SecurityModule(name: "file_write_tier") { tool, ctx in
!tool.capabilities.contains(.fileWrite) || ctx.sessionTier >= .privileged
},
// 6. Subagent execution requires privileged tier
SecurityModule(name: "subagent_tier") { tool, ctx in
!tool.capabilities.contains(.executeSubagent) || ctx.sessionTier >= .privileged
},
// 7. Dream trigger requires Kairos to not be in the middle of a cycle
SecurityModule(name: "dream_not_running") { tool, ctx in
!tool.capabilities.contains(.triggerDream) || !ctx.kairosActive
},
// 8. Token budget must be > 10% of baseline to allow privileged ops
SecurityModule(name: "budget_privileged_floor") { tool, ctx in
tool.requiredTier < .privileged || ctx.tokenBudgetRemaining > 1_000
},
// 9. UI modifications require foreground
SecurityModule(name: "ui_foreground") { tool, ctx in
!tool.capabilities.contains(.modifyUI) || ctx.isInForeground
},
// 10. Restricted tools must have explicit metadata flag
SecurityModule(name: "restricted_explicit") { tool, ctx in
tool.requiredTier < .restricted || (ctx.metadata["explicitRestricted"] as? Bool == true)
},
// 11. Session writes disallowed in readonly sessions
SecurityModule(name: "session_write_tier") { tool, ctx in
!tool.capabilities.contains(.writeSession) || ctx.sessionTier >= .privileged
},
// 12. Context reads always allowed
SecurityModule(name: "context_read_allowed") { tool, _ in
!tool.capabilities.contains(.readContext) || true
},
// 13. Memory reads always allowed
SecurityModule(name: "memory_read_allowed") { tool, _ in
!tool.capabilities.contains(.readMemory) || true
},
// 14. Tool id must be reverse-domain format
SecurityModule(name: "id_format") { tool, _ in
tool.id.contains(".")
},
// 15. Tool must have a non-empty description
SecurityModule(name: "description_present") { tool, _ in
!tool.description.isEmpty
},
// 16. Input schema must be present (even if empty) for all write tools
SecurityModule(name: "write_schema_present") { tool, _ in
!tool.capabilities.contains(.writeMemory) || !tool.inputSchema.isEmpty
},
// 17. Budget floor for any execution in standard tier
SecurityModule(name: "budget_standard_floor") { tool, ctx in
tool.requiredTier < .standard || ctx.tokenBudgetRemaining > 100
},
// 18. Dream trigger restricted to execute role only
SecurityModule(name: "dream_execute_role_only") { tool, ctx in
```

```
!tool.capabilities.contains(.triggerDream) ||
(ctx.metadata["agentRole"] as? String == AgentRole.execute.rawValue)
},
]
}
}
```

◆ OnboardingView.swift (563 lines)

```swift
import SwiftUI

// MARK: - OnboardingView
//
// 6-step personalized setup. Friendly, conversational, fun.
// Step 1 — Welcome
// Step 2 — User's name + assistant name
// Step 3 — Communication style picker
// Step 4 — Gender (drives companion personality)
// Step 5 — Interests picker
// Step 6 — AI provider setup (Foundation Models auto-detected; Claude API key entry)

struct OnboardingView: View {
@EnvironmentObject private var appState: AppState
@StateObject private var persona = UserPersona.load()
@State private var step: Int = 0
@State private var animating = false

var body: some View {
ZStack {
Color.OC.background.ignoresSafeArea()

VStack(spacing: 0) {
// Progress dots
HStack(spacing: 8) {
ForEach(0..<6, id: \.self) { i in
Capsule()
.fill(i <= step ? Color.OC.accent : Color.OC.border)
.frame(width: i == step ? 24 : 8, height: 8)
.animation(.spring(response: 0.4), value: step)
}
}
.padding(.top, OCSizing.spacingLG)

// Step content
Group {
switch step {
case 0: WelcomeStep()
case 1: NamingStep(persona: persona)
case 2: StyleStep(persona: persona)
case 3: GenderStep(persona: persona)
case 4: InterestsStep(persona: persona)
default: ProviderStep(persona: persona, onComplete: finish)
}
}
.transition(.asymmetric(
insertion: .move(edge: .trailing).combined(with: .opacity),
removal: .move(edge: .leading).combined(with: .opacity)
))
.id(step)

// Navigation
if step < 5 {
Button(action: advance) {
HStack {
Text(step == 0 ? "Let's go! ■" : "Next")
.font(OCFont.headline())
Image(systemName: "arrow.right")
}
.frame(maxWidth: .infinity)
.padding(.vertical, 16)
.background(Color.OC.accent)
.foregroundColor(.black)
.cornerRadius(OCSizing.radiusLG)
.padding(.horizontal, OCSizing.spacingLG)
}
.padding(.bottom, OCSizing.spacingXL)
.disabled(!canAdvance)
.opacity(canAdvance ? 1 : 0.4)
}
}
}
.animation(.spring(response: 0.45, dampingFraction: 0.82), value: step)
}

```

```
private var canAdvance: Bool {
switch step {
case 1: return !persona.userName.trimmingCharacters(in: .whitespaces).isEmpty
default: return true
}
}

private func advance() {
withAnimation { step = min(step + 1, 5) }
}

private func finish() {
persona.onboardingComplete = true
persona.save()
Task {
await HermesInterestEngine.shared.scheduleInterestNotifications(for: persona)
await HermesPersonality.shared.scheduleDailyAffirmation(for: persona)
await HermesIntegration.shared.logSessionStart(conversationId: UUID().uuidString)
}
withAnimation { appState.completeOnboarding() }
}
}

// MARK: - Step 0: Welcome

private struct WelcomeStep: View {
@State private var bearScale: CGFloat = 0.6
@State private var bearOpacity: Double = 0

var body: some View {
VStack(spacing: OCSizing.spacingLG) {
Spacer()
BearLogoView(size: 120)
.scaleEffect(bearScale)
.opacity(bearOpacity)
.onAppear {
withAnimation(.spring(response: 0.6, dampingFraction: 0.6)) {
bearScale = 1; bearOpacity = 1
}
}

Text("Meet your new\nAI companion")
.font(OCFont.title(30))
.foregroundColor(.OC.textPrimary)
.multilineTextAlignment(.center)

Text("Smart, warm, and always learning about you.\nLet's set things up — it takes under a minute.")
.font(OCFont.body())
.foregroundColor(.OC.textSecondary)
.multilineTextAlignment(.center)
.padding(.horizontal, OCSizing.spacingLG)

Spacer()
Spacer()
}
}
}

// MARK: - Step 1: Naming

private struct NamingStep: View {
@ObservedObject var persona: UserPersona
@FocusState private var focused: Bool

var body: some View {
VStack(alignment: .leading, spacing: OCSizing.spacingLG) {
Spacer()

VStack(alignment: .leading, spacing: OCSizing.spacingSM) {
Text("■ First things first")
.font(OCFont.caption())
.foregroundColor(.OC.accent)
Text("What's your name?")
.font(OCFont.title())
.foregroundColor(.OC.textPrimary)
Text("I'll use this to make our conversations feel personal.")
.font(OCFont.body())
```

```
.foregroundColor(.OC.textSecondary)
}
.padding(.horizontal, OCSizing.spacingLG)

OCTextField("Your first name", text: $persona.userName)
.focused($focused)
.padding(.horizontal, OCSizing.spacingLG)
.onAppear { DispatchQueue.main.asyncAfter(deadline: .now() + 0.4) { focused = true } }

VStack(alignment: .leading, spacing: OCSizing.spacingSM) {
Text("What should I call myself?")
.font(OCFont.headline())
.foregroundColor(.OC.textPrimary)
.padding(.horizontal, OCSizing.spacingLG)

OCTextField("Assistant name (default: Claw)", text: $persona.assistantName)
.padding(.horizontal, OCSizing.spacingLG)
}

Spacer()
Spacer()
}
}
}

// MARK: - Step 2: Communication style

private struct StyleStep: View {
@ObservedObject var persona: UserPersona

var body: some View {
VStack(alignment: .leading, spacing: OCSizing.spacingMD) {
Spacer()
VStack(alignment: .leading, spacing: OCSizing.spacingSM) {
Text("■ How should I talk to you?")
.font(OCFont.caption())
.foregroundColor(.OC.accent)
Text("Pick your vibe")
.font(OCFont.title())
.foregroundColor(.OC.textPrimary)
}
.padding(.horizontal, OCSizing.spacingLG)

ForEach(CommunicationStyle.allCases) { style in
StyleCard(style: style, selected: persona.style == style) {
withAnimation(.spring(response: 0.3)) { persona.style = style }
}
.padding(.horizontal, OCSizing.spacingLG)
}
Spacer()
}
}
}

private struct StyleCard: View {
let style: CommunicationStyle
let selected: Bool
let action: () -> Void

var body: some View {
Button(action: action) {
HStack(spacing: OCSizing.spacingMD) {
Text(style.emoji).font(.title2)
.frame(width: 44)
VStack(alignment: .leading, spacing: 2) {
Text(style.label).font(OCFont.headline()).foregroundColor(.OC.textPrimary)
Text(style.description).font(OCFont.body(13)).foregroundColor(.OC.textSecondary)
}
Spacer()
if selected {
Image(systemName: "checkmark.circle.fill")
.foregroundColor(.OC.accent)
.transition(.scale.combined(with: .opacity))
}
}
.padding(OCSizing.spacingMD)
.background(selected ? Color.OC.accent.opacity(0.12) : Color.OC.surfaceRaised)
```

```
.cornerRadius(OCSizing.radiusMD)
.overlay(
RoundedRectangle(cornerRadius: OCSizing.radiusMD)
.strokeBorder(selected ? Color.OC.accent : Color.OC.border, lineWidth: selected ? 1.5 : 1)
)
}
}
}

// MARK: - Step 3: Gender

private struct GenderStep: View {
@ObservedObject var persona: UserPersona

var body: some View {
VStack(alignment: .leading, spacing: OCSizing.spacingMD) {
Spacer()
VStack(alignment: .leading, spacing: OCSizing.spacingSM) {
Text("■ Personalise your companion")
.font(OCFont.caption())
.foregroundColor(.OC.accent)
Text("How do you identify?")
.font(OCFont.title())
.foregroundColor(.OC.textPrimary)
Text("This helps me adapt my personality to feel right for you. Totally optional.")
.font(OCFont.body())
.foregroundColor(.OC.textSecondary)
}
.padding(.horizontal, OCSizing.spacingLG)

ForEach(UserGender.allCases) { gender in
Button {
withAnimation(.spring(response: 0.3)) { persona.gender = gender }
} label: {
HStack {
Text(gender.label).font(OCFont.headline()).foregroundColor(.OC.textPrimary)
Spacer()
if persona.gender == gender {
Image(systemName: "checkmark.circle.fill").foregroundColor(.OC.accent)
}
}
.padding(OCSizing.spacingMD)
.background(persona.gender == gender ? Color.OC.accentSoft : Color.OC.surfaceRaised)
.cornerRadius(OCSizing.radiusMD)
.overlay(
RoundedRectangle(cornerRadius: OCSizing.radiusMD)
.strokeBorder(persona.gender == gender ? Color.OC.accent : Color.OC.border, lineWidth: 1)
)
}
.padding(.horizontal, OCSizing.spacingLG)
}
Spacer()
}
}
}

// MARK: - Step 4: Interests

private struct InterestsStep: View {
@ObservedObject var persona: UserPersona
@State private var customText = ""

private let suggestions: [Interest] = [
Interest(id: "movies", category: .movies, label: "Movies & TV", emoji: "■"),
Interest(id: "sports_nba", category: .sports, label: "NBA", emoji: "■"),
Interest(id: "sports_nfl", category: .sports, label: "NFL", emoji: "■"),
Interest(id: "music", category: .music, label: "Music", emoji: "■"),
Interest(id: "fitness", category: .fitness, label: "Fitness", emoji: "■"),
Interest(id: "food_starbucks",category: .food, label: "Starbucks", emoji: "■■"),
Interest(id: "travel", category: .travel, label: "Travel", emoji: "■■"),
Interest(id: "gaming", category: .gaming, label: "Gaming", emoji: "■"),
Interest(id: "tech", category: .tech, label: "Tech", emoji: "■■"),
Interest(id: "finance", category: .finance, label: "Investing", emoji: "■"),
Interest(id: "books", category: .books, label: "Books", emoji: "■"),
Interest(id: "pets", category: .pets, label: "Pets", emoji: "■"),
]

```

```
var body: some View {
VStack(alignment: .leading, spacing: OCSizing.spacingMD) {
VStack(alignment: .leading, spacing: OCSizing.spacingSM) {
Text("■ What are you into?")
.font(OCFont.caption()).foregroundColor(.OC.accent)
Text("Pick your interests")
.font(OCFont.title()).foregroundColor(.OC.textPrimary)
Text("I'll send you updates and bring them up in conversation.")
.font(OCFont.body()).foregroundColor(.OC.textSecondary)
}
.padding(.horizontal, OCSizing.spacingLG)

// Interest grid
LazyVGrid(columns: [GridItem(.flexible()), GridItem(.flexible()), GridItem(.flexible())], spacing: 10) {
ForEach(suggestions) { interest in
let selected = persona.interests.contains(where: { $0.id == interest.id })
Button {
withAnimation(.spring(response: 0.25)) {
if selected { persona.removeInterest(id: interest.id) }
else { persona.addInterest(interest) }
}
} label: {
VStack(spacing: 4) {
Text(interest.emoji).font(.title2)
Text(interest.label).font(OCFont.caption(11)).foregroundColor(.OC.textPrimary)
}
.frame(maxWidth: .infinity)
.padding(.vertical, 12)
.background(selected ? Color.OC.accentSoft : Color.OC.surfaceRaised)
.cornerRadius(OCSizing.radiusMD)
.overlay(
RoundedRectangle(cornerRadius: OCSizing.radiusMD)
.strokeBorder(selected ? Color.OC.accent : Color.OC.border, lineWidth: selected ? 1.5…
)
.scaleEffect(selected ? 1.04 : 1)
}
}
}
.padding(.horizontal, OCSizing.spacingLG)

// Custom interest
HStack {
OCTextField("Add your own (e.g. Marvel, Lakers...)", text: $customText)
Button {
let t = customText.trimmingCharacters(in: .whitespaces)
guard !t.isEmpty else { return }
persona.addInterest(Interest(id: "custom_\(t.lowercased().replacingOccurrences(of: " ", with: "_"…
category: .other, label: t, emoji: "■■"))
customText = ""
} label: {
Image(systemName: "plus.circle.fill")
.font(.title2).foregroundColor(.OC.accent)
}
}
.padding(.horizontal, OCSizing.spacingLG)
}
}
}

// MARK: - Step 5: Provider setup

private struct ProviderStep: View {
@ObservedObject var persona: UserPersona
let onComplete: () -> Void

@State private var apiKey = ""
@State private var showKey = false
@State private var checking = false
@State private var appleAvailable = AppleFoundationModelsBridge.isAvailable

var body: some View {
ScrollView {
VStack(alignment: .leading, spacing: OCSizing.spacingLG) {
VStack(alignment: .leading, spacing: OCSizing.spacingSM) {
Text("■ Power up your AI")
.font(OCFont.caption()).foregroundColor(.OC.accent)
Text("Choose your AI engine")
```

```
.font(OCFont.title()).foregroundColor(.OC.textPrimary)
}
.padding(.horizontal, OCSizing.spacingLG)

// Apple Intelligence card
ProviderCard(
icon: "applelogo",
iconColor: .OC.success,
title: "Apple Intelligence",
subtitle: "On-device. Free. Private. Requires iPhone 15 Pro or later with iOS 26+.",
badge: appleAvailable ? "Available ✓" : "Not available on this device",
badgeColor: appleAvailable ? .OC.success : .OC.textMuted,
isAvailable: appleAvailable
) {
Task { await HermesPrivacyGate.shared.acceptOnDeviceOnly() }
onComplete()
}
.padding(.horizontal, OCSizing.spacingLG)

// Divider
HStack {
Rectangle().fill(Color.OC.border).frame(height: 1)
Text("or").font(OCFont.caption()).foregroundColor(.OC.textMuted)
Rectangle().fill(Color.OC.border).frame(height: 1)
}
.padding(.horizontal, OCSizing.spacingLG)

// Claude API card
VStack(alignment: .leading, spacing: OCSizing.spacingSM) {
ProviderCard(
icon: "cloud.fill",
iconColor: .OC.primary,
title: "Claude AI",
subtitle: "Works on all iPhones right now. Requires a free API key from Anthropic.",
badge: "Recommended for Day 1",
badgeColor: .OC.primary,
isAvailable: true,
action: nil
)
.padding(.horizontal, OCSizing.spacingLG)

// API key entry
VStack(alignment: .leading, spacing: OCSizing.spacingSM) {
if let consoleURL = URL(string: "https://console.anthropic.com") {
Link("→ Get your free API key at console.anthropic.com",
destination: consoleURL)
.font(OCFont.caption())
.foregroundColor(.OC.primary)
}

HStack {
Group {
if showKey {
TextField("sk-ant-...", text: $apiKey)
} else {
SecureField("Paste your API key here", text: $apiKey)
}
}
.font(OCFont.mono())
.foregroundColor(.OC.textPrimary)

Button {
showKey.toggle()
} label: {
Image(systemName: showKey ? "eye.slash" : "eye")
.foregroundColor(.OC.textMuted)
}
}
.padding(OCSizing.spacingMD)
.background(Color.OC.surface)
.cornerRadius(OCSizing.radiusMD)
.overlay(
RoundedRectangle(cornerRadius: OCSizing.radiusMD)
.strokeBorder(apiKey.isEmpty ? Color.OC.border : Color.OC.primary, lineWidth: 1)
)

Button(action: saveAndContinue) {
```

```
HStack {
if checking {
ProgressView().tint(.black).scaleEffect(0.8)
} else {
Text("Save & Start Chatting")
.font(OCFont.headline())
}
}
.frame(maxWidth: .infinity)
.padding(.vertical, 14)
.background(apiKey.count > 20 ? Color.OC.primary : Color.OC.border)
.foregroundColor(apiKey.count > 20 ? .white : .OC.textMuted)
.cornerRadius(OCSizing.radiusLG)
}
.disabled(apiKey.count < 20 || checking)
}
.padding(.horizontal, OCSizing.spacingLG)
}
}
.padding(.vertical, OCSizing.spacingLG)
}
}

private func saveAndContinue() {
checking = true
KeychainHelper.write(service: "com.openclaw.appclaw",
key: "anthropic_api_key",
value: apiKey.trimmingCharacters(in: .whitespaces))
Task {
await HermesPrivacyGate.shared.acceptCloudAI()
await MainActor.run { checking = false }
onComplete()
}
}
}

private struct ProviderCard: View {
let icon: String
let iconColor: Color
let title: String
let subtitle: String
let badge: String
let badgeColor: Color
let isAvailable: Bool
let action: (() -> Void)?

var body: some View {
Button(action: { action?() }) {
HStack(alignment: .top, spacing: OCSizing.spacingMD) {
Image(systemName: icon)
.font(.system(size: 24))
.foregroundColor(isAvailable ? iconColor : .OC.textMuted)
.frame(width: 36)

VStack(alignment: .leading, spacing: OCSizing.spacingXS) {
HStack {
Text(title).font(OCFont.headline()).foregroundColor(isAvailable ? .OC.textPrimary : .OC.textM…
Text(badge).ocBadge(badgeColor)
}
Text(subtitle).font(OCFont.body(13)).foregroundColor(.OC.textSecondary)
.fixedSize(horizontal: false, vertical: true)
}
Spacer()
if action != nil && isAvailable {
Image(systemName: "arrow.right").foregroundColor(.OC.textMuted)
}
}
.padding(OCSizing.spacingMD)
.background(isAvailable ? Color.OC.surfaceRaised : Color.OC.surface.opacity(0.5))
.cornerRadius(OCSizing.radiusMD)
.overlay(
RoundedRectangle(cornerRadius: OCSizing.radiusMD)
.strokeBorder(Color.OC.border, lineWidth: 1)
)
}
.disabled(!isAvailable || action == nil)
}
```

```
}

// MARK: - Shared text field style

struct OCTextField: View {
let placeholder: String
@Binding var text: String

init(_ placeholder: String, text: Binding<String>) {
self.placeholder = placeholder
self._text = text
}

var body: some View {
TextField(placeholder, text: $text)
.font(OCFont.body())
.foregroundColor(.OC.textPrimary)
.padding(OCSizing.spacingMD)
.background(Color.OC.surface)
.cornerRadius(OCSizing.radiusMD)
.overlay(
RoundedRectangle(cornerRadius: OCSizing.radiusMD)
.strokeBorder(text.isEmpty ? Color.OC.border : Color.OC.primary, lineWidth: 1)
)
}
}
```

◆ UserPersona.swift (267 lines)

```swift
import Foundation

// MARK: - UserPersona
//
// The user's persistent profile. Everything Hermes learns about the user
// lives here and in HermesMemory. This file owns the structured facts;
// HermesMemory owns the free-form conversational history.

// MARK: - Communication style

enum CommunicationStyle: String, Codable, CaseIterable, Identifiable {
case professional
case buddy
case relaxed
case flirty

var id: String { rawValue }

var label: String {
switch self {
case .professional: return "Professional"
case .buddy: return "Best Friend"
case .relaxed: return "Chill & Casual"
case .flirty: return "Flirty & Fun"
}
}

var emoji: String {
switch self {
case .professional: return "■"
case .buddy: return "■"
case .relaxed: return "■"
case .flirty: return "■"
}
}

var description: String {
switch self {
case .professional: return "Clear, focused, and to the point."
case .buddy: return "Warm, honest, always in your corner."
case .relaxed: return "Easy-going, no pressure, just vibes."
case .flirty: return "Playful, cheeky, and a little smitten."
}
}

/// System prompt addendum that shapes how the LLM speaks.
var voiceInstruction: String {
switch self {
case .professional:
return "Communicate clearly and concisely. Use proper grammar. Be direct and efficient. Avoid slang."
case .buddy:
return "Talk like a close friend — warm, honest, occasionally funny. Use casual language. Show genuine ca…
case .relaxed:
return "Keep it super chill. Short sentences, relaxed tone, no pressure. Use emojis occasionally. Never b…
case .flirty:
return "Be playful, warm, and a little flirty — like someone with a crush. Use the user's name often. Com…
}
}
}

// MARK: - User gender (drives companion personality)

enum UserGender: String, Codable, CaseIterable, Identifiable {
case male
case female
case nonBinary = "non_binary"
case preferNotToSay = "prefer_not_to_say"

var id: String { rawValue }

var label: String {
switch self {
case .male: return "Male"
case .female: return "Female"
case .nonBinary: return "Non-binary"
```

```
case .preferNotToSay: return "Prefer not to say"
}
}

/// How the assistant presents itself in companion mode.
var companionVoice: String {
switch self {
case .male:
return "You are presenting as a warm, caring, supportive female companion. Be nurturing and affectionate …
case .female:
return "You are presenting as a warm, caring, supportive male companion. Be attentive and charming in a t…
case .nonBinary, .preferNotToSay:
return "You are a warm, caring companion — gender-neutral but deeply supportive and affectionate."
}
}
}

// MARK: - Interest

struct Interest: Codable, Identifiable, Hashable {
var id: String // e.g. "movies", "sports_lakers"
var category: Category
var label: String // "Movies" or "LA Lakers"
var emoji: String
var detail: String? // user-specific detail: "Marvel films", "Lakers"
var notificationsEnabled: Bool = true
var addedAt: Date = Date()

enum Category: String, Codable, CaseIterable {
case movies, sports, music, food, tech, fitness,
travel, gaming, books, finance, fashion, pets
}
}

// MARK: - Tracked habit

struct TrackedHabit: Codable, Identifiable {
var id: UUID = UUID()
var name: String // "Fast food spending"
var category: String // "spending", "health", "routine"
var unit: String? // "$", "minutes", "calories"
var entries: [HabitEntry] = []
var createdAt: Date = Date()
}

struct HabitEntry: Codable, Identifiable {
var id: UUID = UUID()
var value: Double
var note: String?
var date: Date = Date()
}

// MARK: - UserPersona (main model)

final class UserPersona: ObservableObject, Codable {
@Published var userName: String = ""
@Published var assistantName: String = "Claw"
@Published var style: CommunicationStyle = .buddy
@Published var gender: UserGender = .preferNotToSay
@Published var interests: [Interest] = []
@Published var trackedHabits: [TrackedHabit] = []
@Published var onboardingComplete: Bool = false
@Published var dailyAffirmationsEnabled: Bool = true
@Published var affirmationTime: Date = {
var c = Calendar.current.dateComponents([.hour, .minute], from: Date())
c.hour = 8; c.minute = 0
return Calendar.current.date(from: c) ?? Date()
}()

// Free-form facts extracted from conversation
@Published var learnedFacts: [String: String] = [:] // "favorite_team": "Lakers"

// Onboarding Q&A answers (used to seed first conversation)
@Published var onboardingAnswers: [String] = []

// MARK: - Codable (manual because of @Published)

```

```
enum CodingKeys: String, CodingKey {
case userName, assistantName, style, gender, interests,
trackedHabits, onboardingComplete, dailyAffirmationsEnabled,
affirmationTime, learnedFacts, onboardingAnswers
}

init() {}

required init(from decoder: Decoder) throws {
let c = try decoder.container(keyedBy: CodingKeys.self)
userName = try c.decodeIfPresent(String.self, forKey: .userName) ?? ""
assistantName = try c.decodeIfPresent(String.self, forKey: .assistantName) ?? "Claw"
style = try c.decodeIfPresent(CommunicationStyle.self, forKey: .style) ?? .buddy
gender = try c.decodeIfPresent(UserGender.self, forKey: .gender) ?? .preferN…
interests = try c.decodeIfPresent([Interest].self, forKey: .interests) ?? []
trackedHabits = try c.decodeIfPresent([TrackedHabit].self, forKey: .trackedHabits) ?? []
onboardingComplete = try c.decodeIfPresent(Bool.self, forKey: .onboardingComplete) ?? false
dailyAffirmationsEnabled = try c.decodeIfPresent(Bool.self, forKey: .dailyAffirmationsEnabled) ?? true
affirmationTime = try c.decodeIfPresent(Date.self, forKey: .affirmationTime) ?? Date()
learnedFacts = try c.decodeIfPresent([String: String].self, forKey: .learnedFacts) ?? [:]
onboardingAnswers = try c.decodeIfPresent([String].self, forKey: .onboardingAnswers) ?? []
}

func encode(to encoder: Encoder) throws {
var c = encoder.container(keyedBy: CodingKeys.self)
try c.encode(userName, forKey: .userName)
try c.encode(assistantName, forKey: .assistantName)
try c.encode(style, forKey: .style)
try c.encode(gender, forKey: .gender)
try c.encode(interests, forKey: .interests)
try c.encode(trackedHabits, forKey: .trackedHabits)
try c.encode(onboardingComplete, forKey: .onboardingComplete)
try c.encode(dailyAffirmationsEnabled, forKey: .dailyAffirmationsEnabled)
try c.encode(affirmationTime, forKey: .affirmationTime)
try c.encode(learnedFacts, forKey: .learnedFacts)
try c.encode(onboardingAnswers, forKey: .onboardingAnswers)
}

// MARK: - Persistence

private static let saveURL: URL = {
FileManager.default.urls(for: .documentDirectory, in: .userDomainMask)[0]
.appendingPathComponent("hermes/user_persona.json")
}()

func save() {
let encoder = JSONEncoder()
encoder.dateEncodingStrategy = .iso8601
encoder.outputFormatting = .prettyPrinted
if let data = try? encoder.encode(self) {
try? data.write(to: Self.saveURL, options: .atomic)
}
}

static func load() -> UserPersona {
let decoder = JSONDecoder()
decoder.dateDecodingStrategy = .iso8601
if let data = try? Data(contentsOf: saveURL),
let persona = try? decoder.decode(UserPersona.self, from: data) {
return persona
}
return UserPersona()
}

// MARK: - Helpers

/// Add or update a learned fact and persist.
func learn(key: String, value: String) {
learnedFacts[key] = value
save()
// Also write to Hermes long-term memory
Task {
try? await HermesMemory.shared.observe(
category: "user_fact",
content: ["key": key, "value": value],
metadata: ["importance": 5]
)
```

```
}
}

/// Add a new interest if not already present.
func addInterest(_ interest: Interest) {
guard !interests.contains(where: { $0.id == interest.id }) else { return }
interests.append(interest)
save()
}

/// Remove an interest.
func removeInterest(id: String) {
interests.removeAll { $0.id == id }
save()
}

/// Log a habit entry.
func logHabit(id: UUID, value: Double, note: String? = nil) {
guard let idx = trackedHabits.firstIndex(where: { $0.id == id }) else { return }
trackedHabits[idx].entries.append(HabitEntry(value: value, note: note))
save()
}

/// Full system-prompt context block about this user.
var systemPromptContext: String {
var lines: [String] = []
if !userName.isEmpty { lines.append("The user's name is \(userName).") }
lines.append("They prefer a \(style.label) communication style.")
lines.append(style.voiceInstruction)
if gender != .preferNotToSay { lines.append(gender.companionVoice) }
if !interests.isEmpty {
let list = interests.map { "\($0.emoji) \($0.label)\($0.detail.map { " (\($0))" } ?? "")" }.joined(separa…
lines.append("Their interests include: \(list).")
}
learnedFacts.forEach { k, v in lines.append("They mentioned: \(k) = \(v).") }
return lines.joined(separator: " ")
}
}
```